

企业级 Agent 托管平台

参考架构

控制自持 · 算力外租 · 协议解耦

一套企业自持控制平面、按量租用云端算力、以协议而非框架托管 Agent 的架构设计

核心命题

云厂商可以替企业托管机器，却不能替企业回答“谁能跑、谁能干什么、干了什么、花了多少、出了事怎么查”。企业级 Agent 平台要做的，是把不确定的智能行为放进一个确定、可验证、可迁移的边界里。

控制自持

身份、权限、账本、任务状态和接口
契约由企业自己持有，构成控制权本身

算力外租

运行时、沙箱、对象存储和监控面板
按量租用，换一家供应商一样能用

协议解耦

平台与 Agent 之间只约定协议，任何框架通过适配器接入，契约不随框架变化

作者

Yuting

版本

v1.0

日期

2026 年 9 月

文档定位

参考架构与设计提案

本文描述目标架构、设计约束与分阶段交付边界，不声明既有生产能力、性能水平或商业服务等级。

摘要

企业里的 AI Agent 正在从零散试验走向统一托管。决定一个 Agent 能否进入生产环境的，往往不是模型本身，而是围绕它的一整套问题：它以谁的身份行动、能碰哪些系统、密钥放在哪里、任务跑到了哪一步、花了多少钱、出了事怎么查、换供应商要付出什么代价。如果这些能力跟着某一家云上运行时一起建设，企业得到的是更快的上线速度，失去的是对身份、策略、密钥和历史事实的控制权。

本文提出一套通用的企业级 Agent 托管平台参考架构。它是一份设计提案，不是实施结果报告。核心做法有四条：企业自己保留控制平面（身份、权限、账本、任务状态和接口契约），把机器、沙箱、对象存储和监控面板按量租用；平台与被托管的 Agent 之间只约定协议，不绑定任何开发框架；Agent 对外部世界的每一次操作——调用模型、调用工具、执行代码——都必须经过平台的统一出口；所有运行事实都记入企业自己持有的账本。

在业界已有的托管运行时、工作负载身份、凭证保管、网关、沙箱和持久执行能力之上，本文重点补充了几项企业侧机制：用四个相互独立的维度描述 Agent 的能力声明；由平台分配事件序号的最小容器契约；包含用户委托、调度主体和硬性禁止项的权限交集规则；供应商失联时仍然生效的三道停机闸门；由检查点、幂等台账和结果固化组成的“等效恰好一次”副作用语义；以及带哈希链、支持合规删除的企业侧事实账本。

关键词：企业级 Agent；Agent 托管平台；控制平面；工作负载身份；工具治理；统一账本；沙箱；云中立

阅读路线

本文篇幅较长，不同读者可以按下表选读。第一次接触 Agent 平台的读者，建议先看附录 A 的术语表，再从第 1、2、3 节读起，然后跳到第 9 节“一次调用的完整过程”，那一节把全文的机制串在一个具体例子里。

读者	建议章节
管理者、决策者	第一部分（命题与对象）、第 6 节总体架构、第 20 节交付路线、第 23 节结论
平台架构师	全文；重点是十条不变量、三层接口、运行时语义、身份与权限、架构决策
实施团队	第二、三部分和附录 B、C：接口、运行、身份、账本与验收
安全与审计评审	第 13 节威胁模型、第 14–16 节、第 22 节残余风险
第一次接触的读者	附录 A 术语表 → 第 1–3 节 → 第 9 节

目录

I：命题与托管对象	1
1 执行摘要	1
2 问题定义：企业缺的不是容器	2
3 平台托管什么：Agent 的定义与四种形态	4
4 设计方法：原则、不变量与建设边界	8
5 相关工作与本文定位	11
II：体系架构与运行契约	12
6 总体架构：控制平面自持，执行平面可替换	12
7 Agent 的能力声明与准入	15
8 三层接口：对外功能完整，对内约定精简	18
9 一次调用的完整过程	21
10 运行时语义：状态、通道与执行事实	22
11 可靠性与安全管理：熔断、幂等、恢复与停机	25
12 四种形态各自怎么实现契约	27
III：安全治理与运行控制	30
13 威胁模型与信任边界	30
14 身份与权限：Agent 是第一类主体	31
15 公共服务层：副作用的统一出口	33
16 统一账本、计量与可观测性	35
17 权威存储底座	37
18 执行平面与云供应商边界	38
IV：产品化、交付与演进	40
19 企业系统对接与通用化	40
20 第一阶段交付、演进路线与验收	41
21 关键架构决策与取舍	43
22 残余风险与后续规范	43
23 结论	44
A 术语表	45
B L2 容器契约速查	46
C 十条不变量验收清单	47
D 修订记录	47

插图

1 四种 Agent 形态的划分谱系	5
2 形态选择路径	7
3 平台六层总体边界	12
4 总体架构全景	14
5 控制面的六个模块	16
6 能力声明的四个维度	17
7 三层接口的职责分离	18
8 入口层的三个模块	19
9 事件先记账再分发	21

10	一次执行的九个步骤	21
11	数据面的七个模块	22
12	五条行动通道与状态服务	24
13	执行的状态机	24
14	三道停机闸门	26
15	代码型 Agent 的三层实现	28
16	两种令牌的接口隔离	32
17	一次工具调用中凭证的流转	33
18	公共服务层的四个模块	34
19	环境交互类工具的调用链路	34
20	计量三本账	36
21	权威存储底座的四类数据	37
22	执行平面与云端资源	38
23	企业内部系统的对接点	40
24	三个阶段的演进路径	42

表格

1	云运行时与企业平台的职责分工	3
2	平台的价值：各受益方得到什么	4
3	本方案的目标与非目标	4
4	四种形态的对照	7
5	八条设计原则	9
6	十条系统不变量及其验证方式	10
7	四种建设方式	10
8	行业已有能力与本文不重复宣称的内容	11
9	控制平面五个分区及判据	13
10	模块建设矩阵	15
11	能力声明的四个维度	16
12	L1 统一 API 的资源组	19
13	L2 契约的组成	20
14	两类事件的持久化等级	20
15	执行环境、对话线程与执行的区别	23
16	五条行动通道及各自的治理	23
17	四类状态的权威归属与恢复含义	25
18	两类幂等	25
19	中断与恢复的三层交付	27
20	四种形态实现契约的方式	27
21	威胁模型	30
22	四类主体	31
23	两种令牌	32
24	各类事实的权威来源	35
25	执行平面的两条产品线	39
26	供应商组件的三档取舍	39
27	企业内部系统的对接方式	40
28	内核、适配包与策略包	41
29	三个阶段的范围	42
30	第一阶段的六个验收闭环	42
31	关键架构决策	43
32	残余风险	44

PART I

命题与托管对象

先回答三个问题：这个平台为什么要存在，它托管的到底是什么，以及四类 Agent 是怎么划分出来的。

1

执行摘要

一个企业级 Agent 平台要能回答五个问题：**谁能跑、谁能干什么、干了什么、花了多少、出了事怎么查**。这五个问题对应四类平台能力：可控、可计量、可观察、可审计。本文全部的设计都是为了让这四类能力在企业自己手里成立。

云厂商提供的 Agent 运行时负责把请求投递进容器、维持实例、自动伸缩、按“干活／等待”计费。这些事它做得很好，但它并不知道这个容器里的 Agent 是谁、代表谁在行动、被允许碰哪些系统、这次任务算谁的钱、里面发生了什么。它管的是容器，不是 Agent。所以“直接采购云上的 Agent 全套产品”并不会自动补齐企业的治理缺口；相反，它可能把身份、策略、密钥和历史事实一起锁进供应商。

本文的方案建立在六个判断上：

1. **控制平面由企业自己持有**。注册、身份、权限、任务状态、账本和云适配层是长期控制点，不能外包。
2. **执行平面可以按量租用**。机器、沙箱、对象存储和只读的监控面板不构成决定权，换一家供应商一样能用。
3. **平台绑定协议，不绑定框架**。新的 Agent 开发框架通过适配器接入，平台与容器之间的契约不随框架变化。
4. **能力由声明和实测共同决定**。Agent 的能力声明只是申请，通过符合性测试后才生效。
5. **所有副作用都经过统一出口**。调用模型、调用工具、执行代码，没有绕开平台的路径。
6. **所有事实进入同一本账**。管理员的操作和 Agent 的行为使用同一套身份链、同一个事件序列、同一套审计规则。

用一句话概括这套架构：企业自持的控制平面决定 Agent 的身份、版本、能力和责任；一份最小的协议连接可替换的云上执行资源；统一的出口限制 Agent 对外部的每一次行动；一本账保存全部权威事实。

这套架构给企业带来的，是四件用平台以外的手段很难同时得到的东西：接入快——存量 Agent 改几行就能托管，问答类 Agent 填提示词就能上线；用得起——机器按量租、等待不花钱、每次执行有硬上限；管得住——每一次对外行动都经过能记录、能拒绝的出口，凭证从不进入 Agent；说得清——任何一次执行都能回放到人、具体的授权、具体的花费，并且换一家云供应商这些记录一条都不丢。

第一阶段只交付两类 Agent（问答型与代码型）、单一云供应商、定时触发和简版控制台。但有几件事必须从第一天做对，因为事后修改的代价是重写全部已上线的 Agent：容器契约的事件格式、权威状态的归属、模型与工具的统一出口、每次执行的硬上限、两类幂等、令牌的接口隔离、用户身份的验签、供应商失联时仍然生效的停机机制，以及账本的哈希链。流程型 Agent、中断与恢复、人工审批、长期记忆、多云和自主型 Agent 在契约稳定后按第二、第三阶段演进。

本文的组织如下。第一部分回答平台为什么存在、托管的对象是什么、设计遵循什么原则，并说明本文与业界已有工作的关系。第二部分是体系架构与运行契约：总体架构、能力声明与准入、三层接口、一次调用的完整过程、运行时语义、可靠性与安全控制、四种形态各自怎么实现契约。第三部分是安全治理：威胁模型、身份与权限、副作用的统一出口、统一账本、权威存储、执行平面与云供应商边界。第四部分讨论产品化与交付：企业系统对接与通用化、分阶段交付与验收、关键决策、残余风险。附录提供术语表、契约速查和验收清单。

2 问题定义：企业缺的不是容器

2.1 Agent 和传统服务的区别

传统的 API 服务，路径是写死的：请求进来，走固定的代码分支，调用固定的下游，消耗大致可预测的资源。Agent 不一样。它会根据上下文自己决定下一步做什么：选哪个模型、调哪个工具、要不要执行一段代码、要不要先停下来等人确认。同一个 Agent 面对不同输入，走的路径可能完全不同。

这带来一种新的风险。传统服务出问题，通常是“答错了”；Agent 出问题，可能是“做错了”——它能以真实身份对真实系统产生副作用：下一笔订单、发一封邮件、改一条记录、执行一段脚本。所以治理 Agent 的重点，从“它说了什么”转向“它能做什么、做了什么、谁授权的”。

2.2 各团队各自为战会发生什么

如果企业允许各个团队各自持密钥、各自选框架、各自上云，很快会出现四种断裂：

- **身份断裂**。机器身份、开发者身份和最终用户身份混在一起，事后无法证明一次操作到底是谁代表谁做的。
- **治理断裂**。上线检查、工具授权、预算限制和隔离策略散落在各团队的代码和各家供应商的控制台里，没有统一的执行点。
- **事实断裂**。模型、工具、运行时和文件各有各的日志，格式不同、时间不对齐，无法还原一条完整的责任链。
- **供应商断裂**。关键状态和专有 API 沉淀在云端，想换供应商时发现要重新实现整个平台。

这四种断裂都不是靠“再买一个更好的运行时”能解决的，因为它们发生在运行时之上、企业自己的边界之内。

2.3 为什么是现在

三件事同时发生，使这个问题从“可以以后再说”变成“现在必须回答”。

第一，Agent 正在从试验走向生产。当它只是内部试用的聊天助手时，出错的代价是一个不满意的回答；当它开始接触客户数据、内部系统和真实交易时，出错的代价是一次真实的副作用。治理的需求随接触面的扩大而出现，而不是随模型能力的提高而出现。

第二，云厂商开始成套地提供 Agent 基础设施——运行时、沙箱、身份、网关、记忆、观测打包在一起。这些产品单独看都很好用，但如果企业把身份、策略、密钥和历史事实一并放进某一家的产品线，几年后想换供应商时会发现要重写的不是一个模块，而是整个平台。决定“哪些自己持有、哪些可以租”的时间窗口，就是现在。

第三，审计与合规的要求正在具体化。监管方、内审和风控部门开始问同一个问题：这个自动化的动作是谁授权的、依据什么做的、能不能复现。这个问题无法靠事后拼接各家日志来回答，只能靠一开始就设计好的责任链。

2.4 云运行时和企业平台各管什么

表 1 把两者的职责放在一起对比。左列是云运行时通常提供的，右列是企业平台仍然要自己负责的。

表 1 云运行时与企业平台的职责分工

能力	云运行时通常提供	企业平台仍需负责
计算生命周期	部署、伸缩、存活探测、按忙闲计费	哪个 Agent、哪个版本、为什么在运行
身份	实例身份或云账号身份	用户、服务、调度器、Agent 的身份，以及“代表谁”的身份链
权限	云资源的访问控制	能用哪些模型和工具、能碰哪些数据、风险等级、硬性禁止项
凭证	密钥产品或环境变量	凭证对 Agent 不可见、按调用注入、每次取用有记录
状态	容器或会话的临时状态	对话历史、执行进度、检查点、产物，以及它们之间的权威对应关系
观测	机器日志和资源指标	业务事件、策略判定、成本归属、责任回放
迁移	供应商自己的导出功能	稳定的契约、企业侧保存的状态、适配器和退出演练

云厂商管容器，企业平台管 Agent。容器“活着”不代表 Agent “被允许行动”；供应商账单“准确”也不代表成本“能够归到某个团队”。

2.5 平台带来什么

四类断裂的反面，就是平台的价值。表 2 按受益方列出：每一方得到什么，靠本文的哪些机制得到。

表 2 平台的价值：各受益方得到什么

受益方	得到什么	靠什么机制
业务团队	填提示词就能上线一个问答 Agent；不用自己管机器、密钥和权限	共享运行壳池、能力声明与准入、统一 API
开发团队	存量 Agent 改几处就能托管，任何框架都可以；上线有版本、可回滚	协议壳与框架适配器、云端构建、注册与发布
安全与审计	每一次对外行动都经过可记录、可拒绝的出口；凭证从不进入 Agent；任何执行可回放到人	五条行动通道、凭证保险库、权限交集、统一账本与哈希链
财务与成本	费用能归到租户、Agent 和单次执行；失控有上限；账单能与供应商核对	计量三本账、熔断、忙闲权威判断
平台与管理层	换云供应商不重写平台；停掉失控的 Agent 不依赖供应商；平台能推广到其他业务线	容器契约、云适配层、三道停机闸门、内核与策略包分离

2.6 目标与非目标

明确不做什么，和明确做什么同样重要。表 3 列出本方案的边界。

表 3 本方案的目标与非目标

	本方案做	明确不做
平台	打通开发、上线、运行、治理、追责的完整闭环	建设一个覆盖所有场景的“万能自主智能体”
框架	租户用任何框架写 Agent，通过协议接入	要求租户统一使用某一种开发框架
算力	按量租用运行时与沙箱	自建有状态的计算集群
模型与工具	统一出口、统一治理、统一计量	自研基础模型，或替代企业已有的工具系统
多云	接口先行、单云落地、迁移可验证	第一天就维护多套等价的生产环境
常驻任务	事件触发后每次创建一个独立的执行	把常驻守护进程定义为一种新的 Agent 形态

3 平台托管什么：Agent 的定义与四种形态

3.1 什么算一个可托管的 Agent

本文所说的 Agent，不是一个模型、一段提示词或一个聊天页面，而是一个**可以版本化、可以赋予身份、能在一次执行中根据输入和上下文选择模型或工具并产生结果的软件单元**。一个东西只有同时满足下面六项，才进入平台的托管边界：

1. **有定义载体**。它由提示词、声明式流程图、源代码或第三方进程之一定义。
2. **有稳定身份**。它有自己的 Agent ID，不借用开发者或机器的账号。
3. **有责任人**。有明确的人类负责人（owner）和所属租户，出了事能找到人。
4. **有不可变版本**。能发布、灰度、回滚，并且能对应到它所运行的“运行壳”版本。
5. **有执行边界**。每次工作构成一个有开始、有终态、有预算、可以取消的执行（Run）。
6. **有运行物证**。事件、工具副作用、成本和产物都能进入统一账本。

这个定义不评价 Agent 有多“聪明”。模型负责生成内容，Agent 负责决策和编排，平台负责限制它的身份、资源、副作用和责任边界。平台治理的对象是“一个能代表某个主体采取行动的软件单元”，而不是智能程度。

3.2 四种形态怎么划分：看执行循环由谁决定

一个 Agent 运行起来就是一个循环：看当前状态，决定下一步，执行，再看状态。四种形态只按一个问题划分：**这个循环的控制流由谁决定？**答案只有四种可能——平台内置的通用循环、租户声明的流程图、租户自己写的代码、第三方进程自己的规划。这个划分和模型强弱、业务价值、风险高低都无关，因此是完备的：不存在第五种“谁来决定循环”的情况。

这里先定义一个后文反复出现的词。**运行壳 (harness)** 指平台自己实现、负责调用模型和编排工具的那个通用执行循环。运行壳是平台代码；租户只通过稳定的协议提交输入、接收事件或声明能力，不修改运行壳本身。

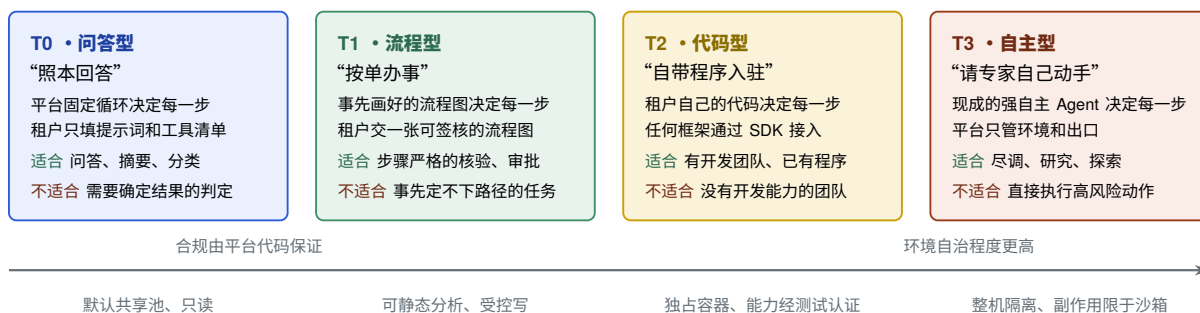


图 1 T0-T3 是“每一步由谁决定”的谱系，不是能力或成熟度的等级

图 1 从左到右，平台对循环的掌控越来越弱，Agent 所处环境的自治程度越来越高。这决定了平台能用什么手段管它：T0 的循环是平台代码，违规在结构上就不可能发生；T1 的流程图是数据，可以在上线前静态分析；T2 的代码是程序，只能通过隔离和统一出口从外面约束；T3 的第三方进程连代码都不在平台手里，只能靠整机隔离和受控出口。

3.3 四种形态分别是什么、适合什么

先用一句话记住四种形态的区别：问答型是“照本回答”，流程型是“接单办事”，代码型是“自带程序入驻”，自主型是“请专家自己动手”。区别的核心只有一点——Agent 的下一步做什么由谁说了算：平台固定的循环、事先画好的流程图、租户自己的代码，还是第三方 Agent 自己的判断。谁说了算，决定了平台能用什么手段管它，也决定了它适合什么业务、不适合什么业务。

T0 · 问答型 Agent：照本回答

是什么：平台准备好一套固定的“看问题、想一想、查工具、给回答”循环，租户只负责填提示词、选模型、勾工具，不写代码。

你要提供：提示词、模型策略、工具清单。

适合：回答错了可以纠正、不产生不可逆后果的场景；数量多、每个都不复杂的场景。

不适合：对结果确定性要求高的场景，例如风控判定、合规审查、资金操作——提示词保证不了某个步骤不被跳过，也保证不了同一个问题两次回答一样。

T1 · 流程型 Agent：接单办事

是什么：业务把步骤、分支和人工审核点画成一张流程图，平台的流程执行器逐步执行，每一步都有记录、都能检查。

你要提供：一张可以静态检查的流程图（画布保存，或从自主型的探索轨迹提炼），以及每个节点用到的工具和模型。

适合：执行顺序有严格要求、某些步骤必须不可跳过、需要风控或合规签核后才能上线的业务。

不适合：步骤事先定不下来的探索性任务；也不能当通用编程工具用——流程语言刻意受限，复杂逻辑应该用代码型。

T2 · 代码型 Agent：自带程序入驻

是什么：租户用任何框架写好的 Agent 程序，通过 SDK 接入平台；循环由租户代码控制，平台在外面管出口和账本，不看代码内部。

你要提供：源代码和依赖；存量程序改三处指向（模型、工具、状态）并换一个入口。

适合：有自己的开发团队、已经有业务逻辑或标准操作流程的代码、需要自定义控制循环的团队。

不适合：没有开发能力的业务团队；也不能把“确定性”寄托在代码自觉上——高风险副作用仍然要经过工具网关，平台不信任容器自己上报的副作用和用量。

T3 · 自主型 Agent：请专家自己动手

是什么：把一个现成的强自主 Agent（Claude Code、Codex 这类命令行 Agent 及同类产品）放进完整的隔离环境，让它自己规划、自己执行；平台只管它的环境和出口。

你要提供：任务描述、环境声明（软件、数据、网络）、选用的第三方 Agent。

适合：目标清楚但路径不清楚、需要在真实环境里试错、单次任务价值高的工作。

不适合：直接执行高风险副作用——它的产出应经过“探索、提炼成流程图、签核”变成流程型再进生产；也不适合大批量、低单价的重复任务，整机隔离不划算。

卡片讲的是每种形态的要点；典型场景、谁来用、平台怎么管、什么时候交付，在表 4 里并排给出。放在一起看，选型的判断依据就清楚了：先看你能交给平台什么（提示词、流程图、代码，还是一个现成的 Agent），再看业务对结果确定性的要求，最后看团队有没有开发能力。

选型时最常见的三个错误，也顺便在这里说清。第一，用问答型做需要确定性的事：把风控判定、合规审查写成提示词交给问答型 Agent，看起来上线最快，但提示词保证不了步骤不被跳过、同一输入不会给出不同结论，这类业务应该用流程型。第二，让自主型直接碰生产系统：自主型适合探索和形成结论，它跑通的路径应该提炼成流程图、经过签核后再以流程型进入生产，而不是让它直接下单、改配置。第三，把流程型当编程语言：流程语言刻意只有六种节点，就是为了能静态检查、能签核，复杂逻辑硬塞进流程图会失去这个好处，应该用代码型。

要特别说明：T0 到 T3 不是从低级到高级。它们表达的是控制流的归属和治理的成本。风险越高的业务，通常越应该向流程型这种确定性形态收敛，而不是追求更高的自治程度。一个处理资金的流程用流程型写成可签核的流程图，比让自主型的模型自由发挥要安全得多。

表 4 四种形态的对照

维度	T0 问答型	T1 流程型	T2 代码型	T3 自主型
下一步由谁决定	平台固定循环	事先画好的流程图	租户自己的代码	现成的强自主 Agent
你交给平台什么	提示词、工具清单	流程图	源码与依赖	任务、环境、第三方 Agent
需要开发能力吗	不需要	业务与风控定义流程，开发提供工具	需要	不需要写代码，需要会用 Agent
结果的确定性	低，每次可能不同	高，步骤可证明	取决于代码	低，开放式
典型场景	客服、摘要、翻译、分类	核验、复核、报表、审批链	研究分析、策略、存量应用	尽调、数据分析、探索研究
不适合	风控判定、资金操作	探索性任务、复杂逻辑	无开发团队	直接执行高风险动作；批量低价任务
谁来用	业务团队自助	业务与风控定义流程，开发提供工具	有开发团队的业务线、量化与研究团队	研究、分析与工程团队
平台怎么管	循环即平台代码	上线前静态分析	管出口，不管内部	整机隔离加受控出口
交付阶段	第一阶段	第二阶段	第一阶段	第三阶段

3.4 怎么选形态

图 2 给出一条选择路径，按顺序问三个问题即可。

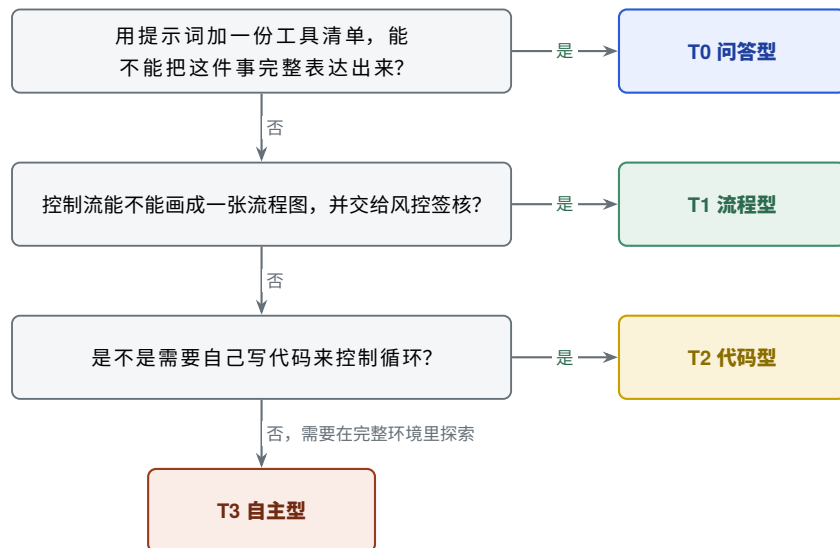


图 2 从业务需求到 Agent 形态的选择路径：按顺序问三个问题

一句口诀：能写成表单的用 T0，能画成流程图的用 T1，必须写代码的用 T2，连流程图都画不出来的用 T3。

3.5 两个常见的追问

用代码写的流程图算 T1 还是 T2？算 T2。判断标准是循环由谁持有：用 Python 等语言写出来的图，运行在租户自己的进程里，是程序而不是数据，平台无法在上线前静态分析它。只有由平台的流程执行器运行、用声明式文件描述、可以静态分析和签核的图，才是 T1。

多个 Agent 协作算什么形态？多 Agent 是 Agent 之间的拓扑关系，不是新形态。平台的一等托管对象有三类：Agent、工作流和工具服务（MCP Server）。一个 Agent 调用另一个 Agent，走的是“把对方注册为一个工具”的路径，沿用工具治理的全部规则（见第 10 节）。

3.6 形态之间的迁移

四种形态之间存在有意设计的迁移路径。T3 适合探索：让模型在完整环境里试出一条可行路径。但高风险、需要重复执行的路径不应该一直停留在 T3，而应该经过“探索 → 提炼流程草稿 → 编译检查 → 人工签核”变成一张 T1 流程图，之后生产上跑的永远是签核过的图。反过来，T2 里稳定下来的共性循环也可以下沉成 T0 运行壳的内置能力，让更多团队用配置的方式复用。

4

设计方法：原则、不变量与建设边界

全文的设计可以归结为三个理念。第一，**把不确定的行为放进确定的边界**：模型的输出不可预测，但它能接触到什么、能做什么、做了什么被记在哪里，这些边界是确定的，治理的对象是边界而不是模型。第二，**控制权来自权威状态和关键出口，而不是来自自己写了多少代码**：谁掌握“真实发生过什么”的记录、谁守着每次行动必经的出口，谁就有控制权；其余都可以租、可以复用。第三，**绑定协议，不绑定框架**：Agent 的开发框架会不断更迭，平台只冻结与容器之间的协议，让任何框架通过适配器接入。

下面的八条原则、控制权公式、十条不变量和四种建设方式，都是这三个理念的展开：原则说明怎么做决策，公式说明边界切在哪里，不变量说明什么不能违反，建设方式说明每个模块从哪里来。

4.1 八条设计原则

表 5 是全部设计决策的推导起点。后文每一个具体设计，都能回溯到其中一条或几条。

4.2 控制权公式

这套架构对“什么该自己做、什么可以外包”的判断，可以压缩成一个公式：

$$\text{控制权} = \text{权威状态的归属} + \text{关键出口的占位}$$
$$\text{运维代价} \approx \text{自己维护的有状态计算集群的规模}$$

第一行说的是：谁掌握“真实发生过什么”的权威记录，谁守着“每次行动必经”的出口，谁就拥有控制权。第二行说的是：真正沉重的运维负担来自有状态的计算集群——调度、隔离、伸缩、故障恢复。两行放在一起，结论就清楚了：权威状态和关键出口自己持有（它们不重），计算集群交

表 5 八条设计原则

#	原则	工程含义
1	绑定协议，不绑定框架	平台只冻结 HTTP、事件流、工具接入等协议边界；新框架等于一个新适配器，不用重做平台
2	管控性与能力性分开	事件真实、副作用受控、网络边界由平台强制，不依赖容器里的代码自觉；检查点、可中断等能力靠框架配合，配合到什么程度就如实声明到什么等级
3	声明加验证，才算能力	能力声明是申请，不是可信证明；通过符合性测试后声明才生效
4	状态归属写明白	对话、执行、环境、长期记忆四类状态各有权威来源，恢复时听谁的写进契约
5	生成可以不完美，闸门必须确定	大模型可以参与生成配置和流程草稿，但最终由格式校验、策略引擎和编译器这类确定性检查收口
6	对外完整，对内精简	对外的接口提供完整的产品能力；平台与容器之间的约定只做三件事：被调用、报存活、被取消
7	状态和出口自持，算力外租	企业保留权威状态和决定权所在的出口，把有状态计算集群的运维交给供应商
8	Agent 是第一类主体	Agent 与人、服务并列，有负责人、有边界、有物证、有履历、有生命周期

给供应商（它最重）。这不是“全部自研”，也不是“全部采购”，而是按控制权和运维代价各自的分布切一刀。

4.3 十条系统不变量

不变量和功能不一样。功能可以多可以少、可以先做后做；不变量是任何一个实现、任何一家企业的实例都不允许违反的约束，也是交付验收时逐条核对的清单（附录 C）。表 6 列出十条，每条都给出验证方法，保证它们不是口号。

4.4 四种建设方式

每个模块的建设方式只有四种：自研、复用企业已有系统、采用开放标准、租用云服务。表 7 说明各自的适用范围和退出要求。

哪些云服务可以租，有一把统一的尺子：**租力气，不租脑子和钥匙**。一个组件只有同时满足三个条件才适合租用——它不握有“能不能做”的决定权；它不托管企业唯一的权威数据（账本、凭证、身份）；它有行业标准，退出成本接近零。用这把尺子量一遍云厂商的全套产品，能通过的只有运行时、沙箱、对象存储和监控面板。监控面板能租，是因为它只读旁观、不碰账本，而且埋点走 OpenTelemetry 开放标准，换一家只改数据地址；云厂商的身份服务、网关、凭证托管、知识库、记忆服务都过不了这三条，因为它们要么握着决定权，要么托管着权威数据。

表 6 十条系统不变量及其验证方式

#	不变量	验证方式
1	容器里永远没有长期密钥	扫描镜像、环境变量、进程和快照；凭证只由工具网关在调用时短时取用
2	副作用只走统一出口	网络层反向测试：从容器内直连模型、工具或外网必须失败
3	工具调用以工具网关的记录为准	对比容器自报事件与网关事件，审计只采信网关一方
4	账本在企业边界内、只追加、带哈希链	修改任何一条历史事件都会导致链校验失败
5	每次执行都有硬上限	步数、调用次数、模型用量、时长、金额任一超限即被任务引擎终止
6	令牌按接口隔离	用运行令牌访问管理接口、用平台令牌访问运行出口，都被拒绝
7	用户身份必须由身份提供方验签	伪造的“代表某用户”字符串无法进入身份链
8	停机不依赖云供应商	模拟供应商接口不可用，停止签发令牌和网关拒绝仍能让 Agent 失去行动能力
9	事件序号由平台分配，先记账再分发	客户端断线后只凭序号就能从账本连续读回
10	管理动作与 Agent 动作记在同一本账	发布、授权、停用、轮换密钥都能与运行事件关联回放

表 7 四种建设方式

方式	范围	选择理由	退出要求
自研	身份、权限、账本、任务引擎、云适配层	这些构成控制权本身，市场上没有完整现成品	元数据和管理接口不依赖任何供应商
复用	身份提供方、模型网关、工具网关、CI、财务系统	企业通常已有成熟能力，重建成本高且割裂现有流程	通过平台的标准接口做增量对接
开放标准	PostgreSQL、OpenTelemetry、事件与对象存储语义	成熟生态，降低迁移和运维门槛	数据可导出、格式有版本
租用	运行时、沙箱、监控面板、对象存储	需要弹性、按量计费 and 供应商的规模效应	不握决定权、不独占权威数据、有标准出口

5

相关工作与本文定位

截至 2026 年 9 月，企业级 Agent 基础设施已经形成几个清晰的共识，本文的设计建立在这些共识之上，而不是另起炉灶。

从各厂商的公开资料看：AWS 的 AgentCore 把运行时、身份、网关、记忆和观测拆成可组合的服务；Anthropic 的 Managed Agents 提供有状态会话和持久事件历史；OpenAI 的 Agents SDK 覆盖会话、沙箱、工具、凭证库、护栏和追踪；Google 的 Agent Platform 提供托管运行时、会话、记忆库、代码执行、访问控制和观测；Microsoft 的 Entra Agent ID 把 Agent 建模为独立的身份主体，强调责任人（sponsor）和生命周期管理。[1, 2, 3, 4, 5]

在执行可靠性和隔离方面，LangSmith Deployment 和 Temporal 提供持久执行、事件历史、中断恢复和长任务编排；腾讯云开源的 CubeSandbox 代表了用微型虚拟机、快照和受控网络承载 Agent 沙箱的基础设施路线。[6, 7, 8]

2026 年 4 月，Anthropic 和 OpenAI 在同一周各自发布了同构的架构：把负责控制流和模型调用的一层，与负责隔离执行的一层明确拆开，凭证不进入执行沙箱。这说明“控制面与执行面分离、凭证不进沙箱、权威事件在网关侧”已经是行业共识，而不是某一家的独特选择。

表 8 行业已有能力与本文不重复宣称的内容

行业已具备的能力	代表性先例	本文不重复宣称的内容
托管运行与弹性伸缩	AgentCore Runtime、Managed Agents、Google Agent Runtime	把 Agent 部署到托管的计算环境
身份、凭证与工具治理	AgentCore Identity 与 Gateway、Entra Agent ID、OpenAI Vaults	Agent 有独立身份、凭证不进业务代码、工具调用受策略约束
会话、记忆与观测	Google Sessions 与 Memory Bank、OpenAI tracing、LangSmith	保存会话状态、追踪调用链、分析运行质量
持久执行与隔离沙箱	Temporal、LangGraph 与 LangSmith、CubeSandbox、云端代码执行	故障后恢复、等待人工输入、隔离执行代码

本文的定位是补齐上述能力之间的[企业控制边界](#)：这些先例各自解决了一段问题，但把它们组合成一个企业能完全掌控、能整体迁移、能对审计负责的平台，中间还缺一组企业侧的契约和机制。本文的增量具体包括：用四个正交维度描述 Agent 能力并经测试认证；由平台分配事件序号的最小容器契约；包含调度主体、用户委托与硬性禁止项的权限交集规则；不依赖供应商管理接口的三道停机闸门；由检查点、工具幂等台账与结果存档组成的“等效恰好一次”语义；以及带哈希链、支持合规删除、能与供应商账单对账的企业侧事实账本。

PART II

体系架构与运行契约

从控制权的边界进入工程实现：六层总体架构、四维度能力声明、三层接口、运行状态，以及四种形态各自怎么实现契约。

6 总体架构：控制平面自持，执行平面可替换

6.1 六层视图

从使用方到云端资源，平台一共有六层边界（图 3）。读这张图要抓住三条线：企业内部系统不是平台的子模块，而是通过明确接口被复用；平台自己的控制平面是一个整体，由企业持有；控制平面与云上执行资源之间隔着一份稳定的契约，契约之下的东西都可以换。

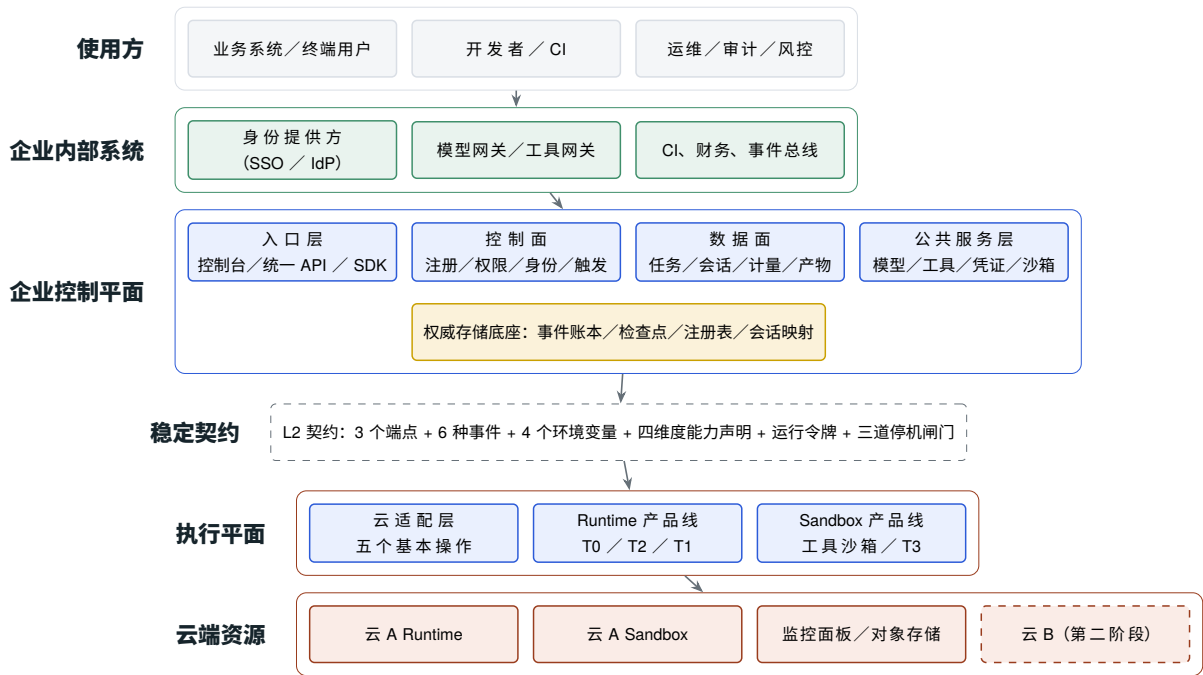


图 3 企业级 Agent 托管平台的六层总体边界。蓝色为企业自持的平台模块，绿色为复用的企业内部系统，黄色为开放标准，橙色为租用的云端资源

从上到下逐层说明：

使用方 四类角色各走各的入口：业务系统通过统一 API 调用 Agent；终端用户通过业务前端使用；开发者通过 SDK 和命令行接入、构建、发布；运维、审计和风控通过管理控制台查看、停用和回放。

企业内部系统 身份提供方、模型网关、工具网关、CI、财务系统和事件总线。平台不重建这些，而是通过标准接口对接（第 19 节）。

企业控制平面 平台自己的全部逻辑，分五个分区（下一小节）。这一层由企业持有，是控制权所在。

稳定契约 平台与每一个 Agent 容器之间的约定。它在第一天冻结，之后只加不改，所以契约之下的东西可以随意替换，契约之上的东西不用返工。

执行平面 Agent 实际运行的地方。云适配层把平台的指令翻译成供应商接口；两条产品线分别承载常驻运行的 Agent 和临时的隔离计算。壳和适配层是平台的，机器是租的。

云端资源 纯租用的机器、沙箱、存储和监控面板。供应商的名字只出现在这一层。

6.2 控制平面的五个分区

控制平面里的模块按五个分区组织，每个分区有一条明确的判据（表 9），任何新模块都能据此判断自己属于哪里。

表 9 控制平面五个分区及判据

分区	判据	核心模块
入口层	人、CI 和业务系统进入平台的入口	管理控制台、统一 API（L1）、SDK 与命令行
控制面	低频的治理动作：定义什么能跑、谁能做什么、何时被唤起	注册与发布、权限管控、身份与访问管理、定时与事件触发、审批工作台
数据面	每一次执行都要经过的高频链路	任务引擎、会话服务、计量与对账、文件与产物、运行监控、中断与恢复
公共服务层	云上的 Agent 调用企业能力时必经的出口	模型网关、工具网关、凭证保险库、沙箱服务
权威存储底座	所有控制与运行事实在企业侧的权威来源	PostgreSQL 事件账本、检查点、注册表、会话映射与哈希链

有两点要解释。第一，入口层和公共服务层为什么要分开：入口层的流量方向是“从外面进来”（人和业务系统调用平台），公共服务层的流量方向是“从云上回来”（正在运行的 Agent 调用模型和工具）。两者方向相反、调用者不同、鉴权方式不同，混在一起会让治理规则互相污染。第二，权威存储底座为什么单独成层：它不是某个模块的附属数据库，而是全部模块共同依赖的事实来源，把它单独画出来，是为了强调“这些数据必须在企业手里”。

6.3 总体架构全景

图 4 把全部模块画在一张图上。这是目标架构的全貌，不区分交付阶段（各阶段的范围见第 20 节）；颜色表示建设方式。读这张图的顺序是从上往下：使用方从入口层进来；企业已有的系统在平台外围被复用；平台自己的控制平面分五个区；一条稳定的容器契约隔开控制平面与执行平面；执行平面里壳是平台的、机器是租的；供应商的名字只出现在最底下的云端资源层。后面的章节会把每一个区域放大：入口层见图 8，控制面见图 5，数据面见图 11，公共服务层见图 18，存储底座见图 21，执行平面与云端资源见图 22，企业内部系统见图 23。

表 10 给出每个模块的一句话职责和建设方式；阶段划分只在这张表和第 20 节出现，架构图本身表达的是目标全貌。



表 10 模块建设矩阵

分区	模块	建设方式	交付期	职责
入口层	管理控制台	自研	一期	查看、管理、回放与一键停用；每个操作进账本
	统一 API（L1）	自研	一期	对话、任务、事件、文件；流式输出与断线续读
	SDK 与命令行	自研 + 租用	一期	少量改造接入、云端构建、发布与符合性检查
控制面	注册与发布	自研 + 租用	一期	不可变版本、灰度、切流、回滚与停用
	权限管控	自研	一期	权限交集、白名单、硬性禁止项、工具风险分级
	身份与访问管理	自研	一期	主体、责任人、身份链、两类令牌、委托验签
	触发与审批	自研	一至二期	定时触发第一期；事件触发与审批工作台第二期
数据面	任务引擎与会话	自研 + 租用	一期	执行状态机、熔断、重试、对话与会话的对应关系
	计量与对账	自研	一期	内部分摊账、供应商账与核对账
	文件与产物	自研 + 租用	一期	引用管理、预签名直传直取、生命周期
	监控与恢复	开放标准 + 租用	一至二期	OpenTelemetry 埋点第一期；中断执行机制与记忆第二期
公共服务层	双网关、保险库、沙箱	复用 + 自研	一期	副作用的统一出口、凭证注入、隔离执行
存储底座	统一账本库	开放标准	一期	事件、检查点、注册表、映射；按时间点恢复
执行平面	云适配层、T0、T2	自研	一期	五个基本操作、运行壳池、代码型 SDK 壳
	T1、T3、多云	自研	二至三期	流程执行器、自主型垫片、跨云重放
云端资源	运行时、沙箱、存储	租用	一期	弹性机器、隔离环境、监控面板和对象存储

6.4 供应商边界规则

供应商的名字只允许出现在云端资源层。控制平面和执行平面中如果需要使用供应商的 SDK，必须被云适配层封装，不能穿透到任务引擎、身份、计量或对外 API。这条规则的意义在于：换供应商时，改动范围被限制在适配层一个模块之内。

7

Agent 的能力声明与准入

这一节对应全景图里的控制面。控制面上的模块都是低频的治理动作（图 5）：注册与发布决定“什么能跑”，权限管控和身份与访问管理决定“谁能做什么”，定时与事件触发决定“何时被唤起”，审批工作台处理需要人介入的高风险操作。这些模块共同回答的问题是“一个 Agent 凭什么被允许运行”，答案就是本节的能力声明与准入规则。



图 5 控制面的六个模块（全景图中的控制面区域）

7.1 为什么不能把一切都绑在形态上

第 3 节按“控制流由谁决定”划分了四种形态。一个自然的做法是把隔离等级、副作用范围、恢复能力也直接绑在形态上：T0 就是共享池加只读，T2 就是独占容器加受控写。这在一家企业内部能用，但作为通用平台会不断遇到例外——代码型 Agent 需要整机沙箱、问答型 Agent 需要浏览器工具、流程型 Agent 需要独占算力。每出现一个例外，就有人提议加“第五种形态”。

本方案的做法是把这些属性拆成四个相互独立的维度，写在每个 Agent 的**能力声明清单（manifest）**里。形态只是四个维度的一组默认值，策略引擎和审计规则按维度执行，不按形态的名字执行。新场景等于新的组合，不是新的形态。

7.2 四个维度

表 11 能力声明的四个维度

维度	取值	含义与由谁决定
循环归属 loop_owner	平台运行壳／平台流程执行器／租户进程／第三方进程	谁实现执行循环，也就决定了谁负责实现容器契约；由定义载体决定
隔离等级 isolation	共享池／独占容器／整机虚拟机	由风险和工具触达范围推导；平台只能上调，租户不能申请下调
副作用范围 side_effect	只读／受控写／沙箱内／外部	由网络策略和工具网关强制，不靠声明
恢复能力 recovery	检查点：每步／每会话／无；中断：冷／热／无	由声明加符合性测试认证

四个维度各有各的“谁说了算”，这是设计的要点。循环归属由定义载体客观决定，租户无法谎报；隔离等级由平台根据风险推导，租户只能接受更严格的等级；副作用范围不看声明，看网络策略和工具网关实际放行了什么；恢复能力可以声明，但要通过符合性测试才算数。四个维度里没有一个是“租户说什么就是什么”。

循环归属	平台运行壳 T0	平台流程执行器 T1	租户进程 T2	第三方进程 T3
隔离等级	共享池 T0	独占容器 T1 T2	整机虚拟机 T3	
副作用范围	只读 T0	受控写（经工具网关） T1 T2	沙箱内 T3	外部 附加条件见 7.3 节
恢复能力	检查点：每步 T1 T2	检查点：无 T0	中断：冷 T1 T3	中断：无 T0 第一阶段默认

方框内的 T0-T3 标记是各形态的默认取值；准入按维度的合法组合表判断，不按形态名字判断。

图 6 能力声明的四个维度及四种形态的默认取值

7.3 合法组合与准入规则

准入不再按形态名字硬编码，而是查一张合法组合表。第一阶段的规则如下：

- 副作用范围为“外部”的 Agent，隔离等级必须是独占容器或以上，并且必须每步保存检查点。理由：能对外部系统产生不可逆影响的 Agent，既要隔离得更严，也要保证失败后能从最近的安全点继续，而不是从头重做。
- 没有检查点的 Agent 只允许持有只读工具集，并关闭平台的自动重试。理由：没有检查点就无法判断上次失败前有没有产生副作用，自动重试可能把一个动作做两遍。
- 在中断执行机制上线（第二阶段）之前，任何声明了中断能力的 Agent 都被拒绝准入。理由：平台还不能正确处理它发出的中断请求，与其运行时出错，不如上线前拦住。
- 高风险副作用路径（例如资金操作）只允许平台流程执行器或租户进程承载，并且必须每步检查点、冷中断（释放机器后从存档恢复）、人工审批。理由：这类路径需要能静态分析或被隔离约束，还要能停下来等人。
- 隔离等级由平台按工具风险自动上调，租户不能申请下调。

这些规则本身也是可以演进的：安全策略、运行实现和产品预设三者独立变化，不需要为每一种新的 Agent 复制一套基础设施。

7.4 为什么不支持常驻 Agent

有一类需求会反复被提出：“让一个 Agent 一直跑着盯盘”“持续监听某个队列”。本方案明确不引入“常驻”形态。原因是常驻进程会破坏一个基本假设——**执行 (Run)** 是计量、审计、取消和重放的基本单位。一个永远不结束的进程，费用没法归到某一次行动，审计没有起止，取消没有安全点。

正确的建模方式是：把“持续监听”变成定时触发或事件触发，每次触发创建一个有边界的执行。这样每一次行动都能单独计费、单独授权、单独追责。定时触发在第一阶段交付，事件触发在第二阶段。

8

三层接口：对外功能完整，对内约定精简

8.1 三层各管什么

平台一共有三层接口（图 7）：面向使用方的统一 API（L1），平台与容器之间的容器契约（L2），容器调用企业能力时经过的服务出口（L3）。三层的设计取向刻意相反：L1 尽可能丰富，让业务系统直接得到产品语义；L2 尽可能薄，让任何框架写的 Agent 都能轻松满足；L3 是统一出口，让容器无论怎么写都绕不开治理。

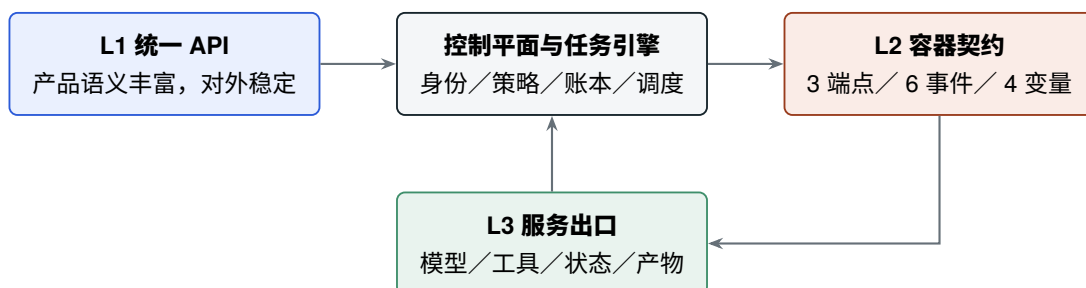


图 7 L1、L2、L3 三层接口的职责分离

三层之间有一条简单的规则：L1 上所有针对数据面的操作，最终落到容器上都只有三种动作——调用它、探测它是否存活、取消它。凡是需要“记住东西”的能力（对话历史、执行进度、审批单据），一律放在平台侧，不放在容器侧。这条定律回答了“某个功能应该做在平台还是做在容器”的问题：容器只做不需要记忆的事。

8.2 L1：面向使用方的统一 API

L1 提供业务能理解的资源，不暴露容器和供应商的概念（表 12）。它是入口层三个模块中面向业务系统的那一个；另外两个——SDK 与命令行、管理控制台——分别面向开发者和运维审计人员（图 8）。三个入口进来的请求，最终都汇到同一个控制平面、同一本账。

L1 有几条横切约束，每条都对应一类事故：

- **创建幂等键。** POST/runs 支持 Idempotency-Key，同一调用方在 24 小时内用同一个键重复提交，返回同一个执行。它防的是：业务系统网络超时后重试，结果创建了两个执行，副作用和费用都变成双份。



图 8 入口层：三类使用方各自的入口（全景图中的入口层区域）

表 12 L1 统一 API 的资源组

资源组	核心操作	关键语义
对话线程 Threads	创建、查询、拉取历史、删除	长期对话记录；历史由平台返回，容器不参与
执行环境 Sessions	查询、主动结束	短期的隔离执行环境，通常由平台隐式管理
执行 Runs	发起、查状态、取消、恢复（第二阶段）、重试	同步流式或异步；执行是所有状态机的主语
事件 Events	按序号续读	客户端只从企业账本读事件，不直连容器
中断 Interrupts	待办列表、结构化单据、决议	第二阶段；决议复用“恢复”进入同一状态机
产物与文件	声明、上传、下载、删除	大对象采用引用式传递和预签名地址，不经过 API 本体
控制面资源	Agent、版本、工具、触发器、用量	注册、发布、授权、触发和用量查询

- 租户强制过滤。所有读接口都以租户为强制过滤条件，不是可选参数。它防的是：一个租户猜到别人的执行 ID 后读到别人的事件。
- 只收平台令牌。L1 只接受发给人、CI 和业务服务的平台令牌，拒绝容器持有的运行令牌（见第 14 节）。它防的是：容器里的代码拿运行令牌去调管理接口改授权。
- 链路追踪标识。所有请求携带标准的链路追踪标识（W3C traceparent），全链路传播。

8.3 L2：不可返工的最小容器契约

L2 是平台最重要的长期资产。它不承载任何产品便利性，只表达“任何 Agent 容器都必须履行的最小义务”。契约的全部内容三个端点、六种事件、四个环境变量和一份能力声明。

```
POST /invoke {run_id, thread_id, input, resume?, context: 只读注入}
    -> 服务器推送事件流 (SSE)

GET /ping -> Healthy | HealthyBusy

POST /cancel {run_id} -> 202
```

三个端点的含义：/invoke 是平台把一次执行交给容器，容器以事件流的方式陆续返回结果；/ping 是平台探测容器是否还活着，“忙”只是一个提示，忙闲的权威判断在平台侧（第 18 节）；/cancel 是平台要求容器在下一个安全点退出，如果容器在宽限期内没有退出，云适配层会直接销毁它。

六种事件把各种框架的内部差异归一成同一套语义（表 13）。任何框架只要能把自己的运行过程翻译成这六种事件，就能被平台托管。

表 13 L2 契约的组成

组成	内容	设计目的
3 个端点	invoke、ping、cancel	被调用、报存活、被取消
6 种事件	message.delta（输出片段）、tool.call（工具调用）、checkpoint（检查点）、interrupt（中断请求）、artifact（产物引用）、run.status（状态与终态）	把框架差异归一为事件语义
4 个环境变量	MODEL_GATEWAY_URL、TOOL_HUB_URL、CHECKPOINT_STORE、ARTIFACT_STORE	只注入平台能力的地址，不注入任何长期密钥
1 份能力声明	四个维度、是否流式、最长运行时长、运行壳版本	声明能力、限制和运行壳版本

四个环境变量要单独说明：它们是平台告诉容器“模型在哪里、工具在哪里、检查点存哪里、产物存哪里”的方式。容器里没有任何密钥，只有这四个地址；绑定当前执行的短时运行令牌不放在环境变量里，而是随每次调用的只读上下文注入，分钟级有效、到期作废。这就是不变量 1（容器里永无长期密钥）的实现方式。

8.4 事件的持久化与分发

容器产生的事件，先由平台记账，再分发给客户端。这样做的目的有三个：客户端断线后可以凭序号从账本续读；容器退出后事件仍然可以回放；事件的顺序由平台统一决定，容器无法伪造顺序。

但“先记账”不能简单理解为每一个输出片段都同步写一次数据库——那会把一次磁盘写入放到每个 token 的流式路径上，流式体验会明显变差。所以不同事件采用不同的持久化等级（表 14）：工具调用、检查点、产物和终态属于事实或控制事件，必须写入主账本成功后才分发；输出片段面向流式体验，分配序号后先写入短期保留的流表并立即分发，再按窗口批量合并成消息记录。

表 14 两类事件的持久化等级

事件类别	分发前的条件	故障时的语义
事实与控制事件	主事件表提交成功	可完整续读和审计，不接受丢失
输出片段 message.delta	短期流表接收并获得序号	允许弱持久；最多丢失尚未合并的片段

容器提供 event_id 用于去重，但不能自行决定序号；序号由平台在接收时分配。容器到平台的链路中断时，该执行进入失败态并标记为可重试，是否真的重试由第 11 节的规则决定。



图 9 事件先记账再分发：事实事件强持久后分发，输出片段经短期流表降低流式延迟

8.5 契约怎么演进

六种事件的格式在第一阶段冻结，但冻结不等于永远不变。演进规则是“只加不改”：新增字段必须是可选的；每个事件带 `schema_version`；平台至少同时读两个版本一段时间。中断事件和恢复字段在第一阶段就定义好，执行机制到第二阶段再实现——这样第二阶段不需要修改任何已上线容器的契约。

8.6 L3：容器调用企业能力的出口

L3 由模型网关、工具网关、检查点存储、产物服务和记忆服务（第二阶段）组成。运行中的容器只能通过 L3 接触外部能力：平台用环境变量注入这些服务的地址，用分钟级有效的运行令牌完成认证。容器不持有管理接口的令牌，也不持有任何长期的工具凭证。第 15 节详细说明这一层。

9

一次调用的完整过程

前面几节分别讲了架构、能力声明和接口。这一节用一个具体的例子把它们串起来：业务系统请求一个 Agent 读取某项业务数据、生成一份报告文件。这个过程不依赖任何特定的 Agent 框架或云厂商。

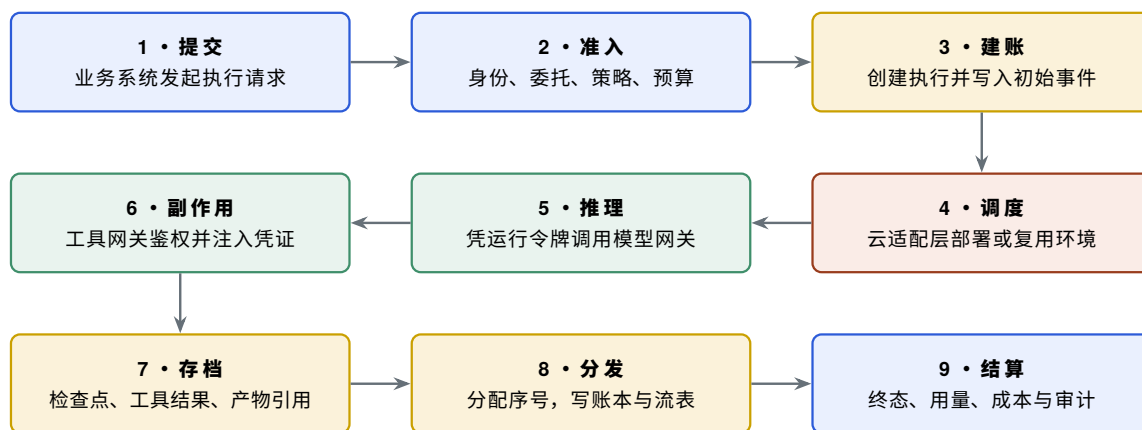


图 10 一次执行从业务请求到事实结算的九个步骤

- 1. 提交。**业务系统携带自己的平台令牌、一个幂等键，以及一个由身份提供方签发的“代表用户”令牌（如果这次调用是替某个用户发起的），调用 `POST/runs`。
- 2. 准入。**统一 API 从已认证的身份解析出租户，验证用户委托令牌的签名，检查这个 Agent 的版本是否可用、工具授权是否有效、是否触碰硬性禁止项、是否超出预算。任何一项不通过，请求在这里就被拒绝，并以“策略拒绝”而不是“系统错误”记入账本。

- 3. **建账。**任务引擎按幂等键创建执行（重复提交返回同一个），把准入结果和初始状态写入企业账本。从这一刻起，这次执行有了序号、身份链和预算上限。
- 4. **调度。**调度器根据能力声明选择执行资源，通过云适配层部署一个新环境或复用现有环境，然后调用容器的 `/invoke`。
- 5. **推理。**容器只持有绑定当前执行的短时刻牌。它需要调用模型时，请求统一经过模型网关；网关检查模型白名单、扣减用量预算，并记录这次调用的 token 数。
- 6. **副作用。**Agent 决定调用一个读取业务数据的工具。工具网关重新计算这个 Agent 此刻的有效权限，检查幂等键，从凭证保险库短时取出所需凭证并代为调用目标系统。容器从头到尾接触不到凭证。
- 7. **存档。**工具结果、模型结果，以及产生副作用前的检查点都被存档。生成的报告文件只在账本里保存内容摘要和对象存储引用，文件本身走对象存储。
- 8. **分发。**事件入口完成去重、分配序号，事实事件写入主账本，输出片段写短期流表，然后向客户端流式分发。客户端断线后凭序号续读。
- 9. **结算。**执行到达终态后，平台汇总模型、工具、运行时、沙箱和存储的用量，生成内部分摊、供应商对账和审计视图。

从这个过程可以看到几件事：容器在整个过程中没有拿到过任何密钥；每一次对外的动作（调模型、调工具）都经过了一个平台能记录、能拒绝的出口；事件的顺序和真实性由平台而不是容器决定；成本在执行结束时就能归到具体的租户和执行。这四点分别对应可控、可审计、可观察、可计量。

10

运行时语义：状态、通道与执行事实

这一节和下一节对应全景图里的数据面（图 11）——每一次执行都要经过的高频链路。任务引擎和会话服务管“这次执行跑到哪、属于哪段对话”，计量与对账管“花了多少”，文件与产物管“产出了什么”，运行监控管“系统健不健康”，中断与恢复和记忆服务是第二阶段加入的两项能力。本节先讲这些模块共同依赖的运行时语义：状态怎么分、通道有几条、执行怎么变化。



图 11 数据面的七个模块（全景图中的数据面区域）

10.1 三个不能合并的概念：执行环境、对话线程、执行

平台用三个概念描述运行时状态，它们经常被混为一谈，但各自管的东西不同（表 15）。可以用一个日常比喻：**执行环境（Session）** 像一次开的独立房间，用完可以关掉；**对话线程（Thread）** 像这位客户与企业的完整往来记录，可以跨很多次到访；**执行（Run）** 像一次具体的办事，有开始、有结果、有费用。

表 15 执行环境、对话线程与执行的区别

	执行环境 Session	对话线程 Thread	执行 Run
是什么	一块隔离执行环境的租约	对话消息的历史	一次有终态的执行
管什么	隔离、资源、内存中的临时状态	多轮对话的上下文连续	进度、事件、重试、成本
生命周期	活跃时占用机器，空闲时释放	长期，跨多个执行环境	分钟到小时
权威来源	平台的会话映射加供应商的租约	企业的会话服务	任务引擎与事件账本

一个对话线程可以跨多个执行环境：每次活跃期绑定一个环境，空闲后环境被释放，下次活跃再开一个。对话线程与执行环境之间的对应关系由企业自己保存（这是权威数据），而“同一个用户的请求尽量落到同一台机器”这类会话保持机制的机械实现可以租用供应商的。

10.2 五条行动通道与一组状态服务

一个运行中的 Agent 要对外部世界产生影响，只有五条通道（图 12）。这五条通道之外，是由网络策略和沙箱构成的墙。只要平台守住这五条通道，即使完全不信任容器里的代码，也能保证副作用、身份、成本和事件都进入统一的治理链。状态与数据服务（检查点、产物、记忆）是支撑这些通道的状态服务，不算行动通道，因为它们本身不对外部世界产生影响。

表 16 五条行动通道及各自的治理

交互	唯一通道	通道上的治理
被调用	平台的调用接口	鉴权、限速、熔断、放置；容器端口不对公网开放
调用模型	模型网关	数据分级、模型白名单、用量预算、计量、黑名单
操作外部系统	工具网关	权限交集、风险分级、参数级风控、幂等、权威事件
请求人工输入	中断单据（第二阶段）	按类型路由、超时处理、审批人和决议进入事件流
调用其他 Agent	把目标 Agent 注册为一个工具	沿用工具治理；委托只能收窄权限
状态与数据	检查点、产物、文件、记忆	租户隔离、过期清理、引用式传递、写入前净化；不算行动通道

这个封闭边界约束的是可识别的接口、网络 and 平台状态通道。域名解析、时序信号、以及通过产物内容影响后续输入这类隐蔽信道，不在本文的威胁模型之内，需要结合出网代理、内容净化、速率限制和独立的安全评估另行处理。

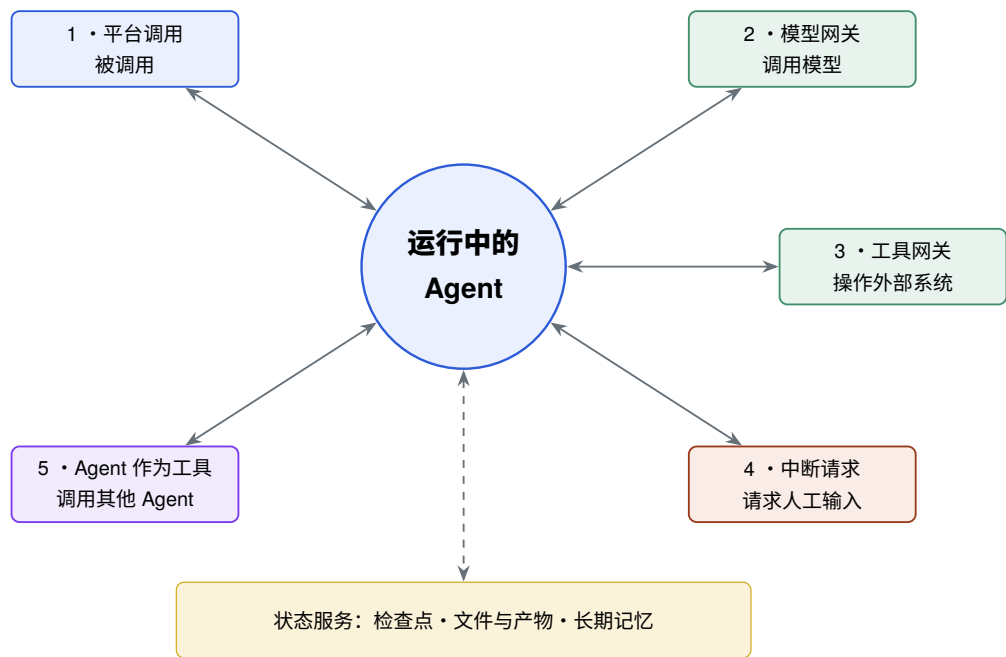


图 12 五条行动通道构成封闭的治理边界，状态服务独立承载恢复与数据

10.3 执行的状态机

每次执行的状态变化很简单（图 13）：排队、运行、然后进入完成、失败或取消三种终态之一；第二阶段加入“等待输入”状态，用于中断与恢复。

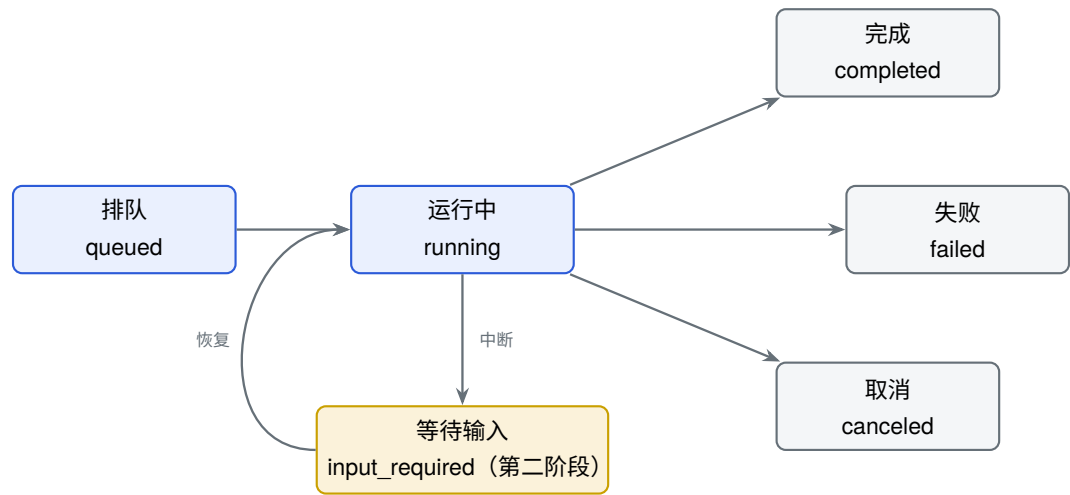


图 13 执行的状态机；策略拒绝和预算超限使用独立的错误类别

失败态带有错误类别，至少包含四种：可重试、不可重试、策略拒绝、预算超限。其中“策略拒绝”要单独说：Agent 被风控拦下来，不是平台故障，而是控制生效的证据。它必须在监控指标、审计记录和调用方看到的错误里与真正的异常区分开，否则运维会把正常的拦截当成故障去“修复”。

10.4 四类状态各归谁管

第 4 节的原则 4 要求状态归属写明白。表 17 是具体的归属，恢复时以此为准。

表 17 四类状态的权威归属与恢复含义

状态	权威归属	恢复时的含义
对话状态	会话服务与对话线程	提供历史的连续性，但不等于恢复执行
执行状态	检查点	决定从哪个安全点继续
环境状态	执行环境的文件系统与产物	需要引用、快照或重建
长期记忆	记忆服务（第二阶段）	不参与执行恢复；写入前净化

11 可靠性与安全控制：熔断、幂等、恢复与停机

这一节讲四个机制。它们共同回答一个问题：当 Agent 失控、重复、卡住或者必须立刻停下来时，平台靠什么保证损失有上界。

11.1 熔断：损失的硬上限

任务引擎为每一次执行设置硬上限：步数、工具调用次数、模型用量、运行时长和估算金额。任何一个维度超限，执行立即转为“预算超限”终态。租户级还有并发数和每日金额的上限，采用可覆盖的默认值；共享池同时设置池级和租户级的并发上限，防止一个租户的突发流量挤占其他租户。

熔断和配额管理是两件事。配额管理包括界面、审批流程、周期性策略，可以在第二阶段建设；熔断是保险丝，必须在第一阶段就有。原因很直接：如果没有熔断，平台会在第一次遇到死循环的 Agent 时才发现，计量系统只能事后解释损失了多少钱，却不能阻止损失发生。熔断的实现成本很低——几个计数器和几次比较——但它是上线首周最可能用到的机制。

11.2 两类幂等要分开

“幂等”指同一个操作重复执行不会产生额外效果。平台需要两类幂等，键不同、解决的问题也不同（表 18）。

表 18 两类幂等

类型	键	解决的问题
创建幂等	调用方作用域内的 Idempotency-Key	网络重试不会创建多个执行
副作用幂等	run_id + step_id + attempt	恢复或重试不会把真实动作做第二遍

自动重试默认最多三次，逐次拉长间隔。但一次失败能不能自动重试，不看容器自己说什么，看工具网关的幂等台账：只有“自上一个检查点以来没有产生过不可确认的副作用”时，失败才算可重试。没有检查点的 Agent 默认关闭自动重试。

11.3 等效恰好一次的副作用

分布式系统无法仅靠一次网络请求保证真实世界里的动作恰好发生一次。本架构用三件事组合出等效的语义：

副作用前先保存检查点 + 工具网关的幂等台账 + 结果存档 $\Rightarrow$ 等效恰好一次

第一件保证失败后有地方可以回到；第二件保证回到之后不会把已经做过的动作再做一遍；第三件保证模型的输出也被存档——恢复时不重新采样，否则同一次执行可能在恢复后走向另一个分支。

11.4 三道停机闸门

停掉一个失控的 Agent，不能依赖云供应商的管理接口在那一刻恰好可用。所以停机路径分三道闸门（图 14）：

- 1. 停止签发。**在注册表里禁用这个 Agent，平台不再为它签发新的运行令牌。
- 2. 切断行动。**模型网关和工具网关按 Agent ID 拒绝它已经拿到的令牌。到这一步，它已经不能调模型、不能调工具，也就不能再对外部产生任何副作用。
- 3. 销毁实例。**云适配层调用供应商接口回收机器。

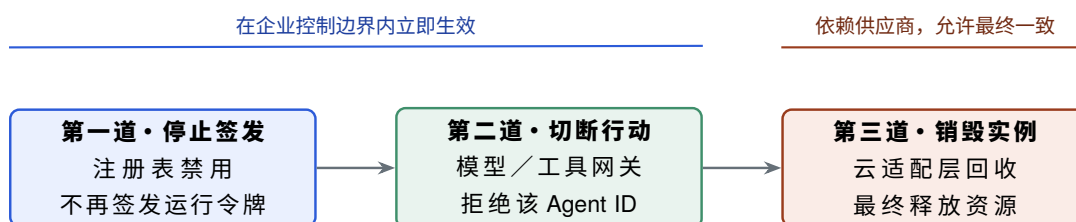


图 14 三道停机闸门：前两道先切断行动能力，第三道完成资源回收

前两道闸门完全在企业控制边界之内。即使第三道暂时失败，Agent 也已经失去了对外行动的能力，剩下的只是一台还在运行但什么都做不了的机器。正常的下线走排空流程：先停止接受新的执行，允许合规的存量执行跑完，关闭挂起的单据，再回收。

11.5 中断与恢复分三层交付

“中断与恢复”指 Agent 在执行中途需要等待——等人审批、等外部系统回调、等另一个 Agent 的结果——这时把状态存好、释放机器，等结果回来再从存档处继续。这个需求不限于某一种形态：代码型 Agent 问人、流程型 Agent 等审批，都会用到。

它被拆成三层，分阶段交付（表 19）。这样拆的理由是：第一层几乎没有成本，但不做就会导致第二阶段修改所有已上线的容器；第二层随 SDK 一起就有，还顺带解决了长任务故障重跑；真

正有工程量的是第三层，而第一阶段的两种形态并不依赖它——问答型的多轮对话是每轮一个执行，不需要执行内部的中断；等待期间的机器成本，供应商按忙闲计费已经覆盖了大半。

表 19 中断与恢复的三层交付

层	阶段	内容
事件格式冻结	第一阶段	中断事件和恢复字段进入容器契约，之后不再改契约
检查点就位	第一阶段	SDK 壳默认配置检查点存储，长任务失败后可以从存档尽力重跑
执行机制	第二阶段	保存检查点、释放机器、路由单据、恢复执行、在新环境中从存档继续

12 四种形态各自怎么实现契约

四种形态面对的是同一份容器契约，但由谁来实现它各不相同（表 20）：问答型和流程型由平台代码实现，合规由平台代码保证；代码型由租户代码通过 SDK 实现，所以需要声明加认证；自主型由平台的垫片替第三方进程实现。

表 20 四种形态实现契约的方式

形态	循环归属	平台的实现方式	关键边界
T0	平台运行壳	共享的无状态运行壳池；按请求注入身份；工具白名单由平台代码决定	版本由能力声明和运行壳共同确定
T1	平台流程执行器	六种节点的流程描述语言；格式、拓扑和策略的静态分析	最终签核的对象必须是声明式流程图
T2	租户进程	HTTP 加事件流的协议壳、框架适配器、网关与网络兜底	能力按恢复、重建、出网三项实测认证
T3	第三方进程	编排器或垫片连接整机隔离环境	垫片不是安全边界；高风险路径应提炼到 T1

12.1 T0：平台运行壳

问答型 Agent 不为每个 Agent 启动独立进程。部署只是在注册表里写一条能力声明，运行发生在共享的运行壳池里。运行壳是无状态的：身份按每个请求注入，系统指令和租户的提示词分段隔离，工具白名单由平台代码决定——租户的提示词只是数据，无论写了什么都不能扩大工具集。

运行壳是全部问答型 Agent 的公共代码，这带来一个必须治理的风险：一次错误的运行壳升级会让数量最多的一类 Agent 同时停摆。所以一个问答型 Agent 的版本不是单一的提示词版本，而是“能力声明版本 + 运行壳版本”的二元组；运行壳升级必须走灰度，注册表允许按租户钉住运行壳版本，运行壳本身也要通过符合性测试。共享池还要设置租户级并发上限，并按活跃时长把供应商的忙时账单分摊给各租户。

12.2 T1：平台流程执行器（第二阶段）

流程型 Agent 使用一种刻意受限的流程描述语言，只有六种节点：工具、模型、人工、分支、并行、子流程。分支条件用确定性表达式，节点的输入输出用 JSON Schema 描述。不图灵完备是有意为之——正因为受限，流程图才能在上线前被静态分析，才能被风控签核。

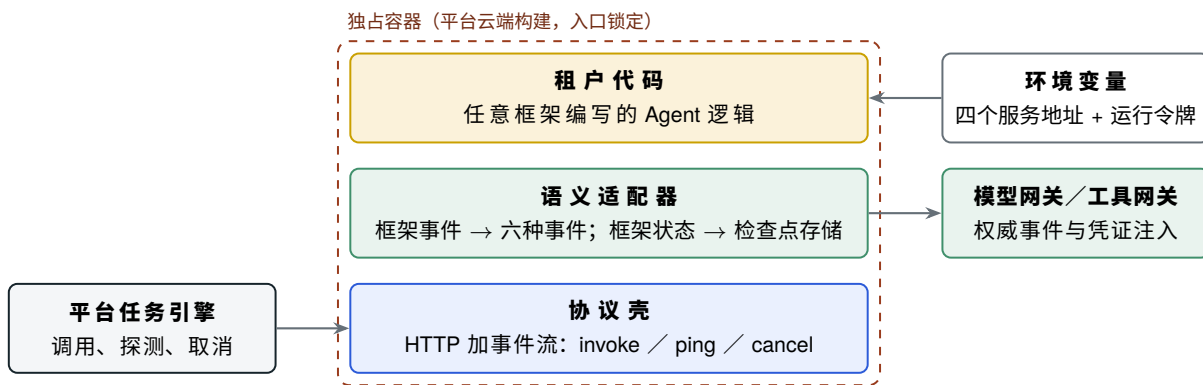
编译分三个阶段：格式校验、拓扑校验、策略静态分析。策略分析里最重要的一项是“支配点检查”：对每一个高风险工具节点，检查所有能到达它的路径是否都必经一个人工审批节点；有一条路径绕过了审批，编译期就拒绝。

平台明令禁止“把代码自动转成流程图后直接运行”。开发者可以写代码，非开发者可以画流程图，机器可以生成流程草稿，但最终签核的对象必须是一张可以静态验证、可以追责的声明式流程图。一张从代码里反推出来、可能残缺的图交给风控签核，等于合规造假。

12.3 T2：租户容器通过 SDK 实现契约

代码型 Agent 的实现分三层（图 15）：

- 1. 协议壳。**与框架无关的 HTTP 加事件流服务，实现三个端点。它把租户的程序从“跑完就退出的脚本”变成“常驻服务”；端口不对公网开放。
- 2. 语义适配器。**把具体框架的运行过程翻译成六种事件，把框架的状态保存机制接到平台的检查点存储。平台官方维护少数主流框架的适配器；框架没有的能力不硬造，如实降级声明。
- 3. 不信任兜底。**这一层与框架无关：权威的工具调用事件来自工具网关，副作用范围由网络策略强制，能力通过实测认证。前两层是给租户的便利，第三层是给平台的保证。



不信任兜底：权威事件在网关侧、副作用由网络策略强制、能力经实测认证，与容器里的框架无关

图 15 代码型 Agent 的三层实现：租户代码、语义适配器、协议壳，外加与框架无关的不信任兜底

存量 Agent 的迁移体验可以概括为“改三处、换一个入口”：模型地址改到模型网关、工具调用改到工具网关、状态保存接到框架的检查点插座，入口由 SDK 壳统一。业务逻辑本身不需要改动。下面是概念性的伪代码，只用来说明迁移边界，不代表真实的 SDK 名称：

```
model.base_url <- MODEL_GATEWAY_URL # 模型指向网关
tools.endpoint <- TOOL_HUB_URL # 工具指向网关
checkpoint.backend <- CHECKPOINT_STORE # 状态接到检查点存储
runtime.agent <- my_graph # 原有业务逻辑不改
runtime.serve() # 由 SDK 壳提供 invoke / ping / cancel 与事件流
```

镜像由平台在云端构建，容器入口由构建流程锁定，租户只能声明依赖和代码目录，不能替换协议壳；壳的版本号随终态事件上报。这不是安全边界（安全边界在网关和网络），而是保证线上行为不漂移：通过了认证的壳，不能在部署时被悄悄换掉。

符合性认证至少包含三个破坏性测试：进程被强制终止后能否按声明恢复；执行环境被回收后能否重建；出网被封锁时是否真的没有旁路。测试套件不属于运行时架构，但它是生产准入不可缺少的交付物。

12.4 T3：编排器与垫片（第三阶段）

自主型 Agent 需要把三个经常被混在一起的东西分开：

- **拓扑**。平台编排器加沙箱内的薄守护进程，或者第三方命令行 Agent 加平台垫片。这是形态定义的内容。
- **产品**。可选的云端开发环境、长期对话和多人协作。这是产品线，不是形态的一部分。
- **治理管线**。探索、提炼流程草稿、编译、人工签核，最终进入 T1。这是让 T3 的产出安全地进入生产的通道。

垫片是楔在平台与第三方进程之间的轻量组件，负责六件事：代管进程生命周期、把第三方的输出翻译成六种事件、把它的权限询问转给平台判定、落地平台的指令、只持有短时令牌、处理异常语义。垫片明确不做的事：不做决策、不负责网络封锁、不解析第三方的业务语义。垫片和不受信的进程跑在同一台机器上，设计上必须假设它可能被攻破，所以它不是安全边界；安全边界是整机隔离、模型网关和受控的网络出口。整机快照必须按租户密钥加密、设置过期时间、恢复前扫描已知的密钥模式，短时令牌不进快照、恢复时重新注入。

PART III

安全治理与运行控制

把不可信的智能行为约束在可验证的边界内：主体身份、副作用的统一出口、审计账本、权威存储，以及与云供应商的隔离。

13

威胁模型与信任边界

先说清楚平台信任谁、不信任谁。平台假设租户代码、模型输出、外部文件和第三方工具的响应都可能是恶意的或被污染的；企业的身份提供方、策略服务、账本写入路径和网关的服务身份属于受控的信任根。云供应商被信任提供合同约定的计算和存储，但不被信任保存唯一的治理事实，也不被信任拥有最终的决定权。

在这个前提下，表 21 列出主要威胁、对应的缓解手段，以及缓解之后仍然需要关注的部分。

表 21 威胁模型

威胁	主要缓解	仍需关注
恶意或被攻破的租户代码	独占容器或微型虚拟机、受控出网、只发短时运行令牌	内核与虚拟化逃逸
提示注入与跨工具的“困惑代理”	工具白名单、权限交集、参数策略、敏感读取与对外发送分离	合法权限的组合被诱导滥用
凭证窃取	凭证只在工具网关短时取用，不进容器、不进模型上下文、不进快照	网关的服务身份被攻破
记忆与文件投毒	来源标记、写入前净化、过期清理、读写权限分离	非结构化内容里的隐式指令
绕过审批或重复副作用	支配点检查、结构化中断单据、工具幂等台账	外部系统本身不支持幂等
供应商故障或锁定	企业自持状态、五个基本操作的适配层、退出演练、三道停机闸门	托管计算能看到运行时的明文
审计抵赖或篡改	平台分配序号、只追加的事件、哈希链、独立导出	信任根或签名密钥失陷
沙箱逃逸与资源滥用	微型虚拟机、资源硬限额、镜像扫描、到期销毁	零日漏洞与隐蔽信道

“困惑代理”指这样一种情况：一个 Agent 合法地同时持有 A 工具的读权限和 B 工具的对外发送权限，被注入的恶意指令诱导后，把从 A 读到的机密通过 B 发了出去。每一步都在权限之内，组

合起来却是泄露。这是行业尚未彻底解决的问题，本文用风险分级和组合授权双签来降低它，但不声称消除它。

本文不声称消除域名解析、时序、功耗等隐蔽信道，也不覆盖模型权重的供应链、芯片级侧信道，或企业身份提供方自身的失陷。这些风险需要由基础设施安全、供应商管理和组织级的零信任体系共同承担。

14 身份与权限：Agent 是第一类主体

14.1 四类主体

平台里有四类主体（表 22）。前三类是“谁在发起调用”，第四类是“被调用的执行单元”。把 Agent 列为独立主体，是整个身份设计的起点：它不借用开发者的账号，也不借用机器的账号，有自己的 ID、责任人和生命周期。

表 22 四类主体

主体	来源	行动模式	权限上界
用户	终端用户，经业务系统委托	代表用户（OBO）	用户自身的权限
服务	业务系统或后端服务	以服务身份	服务账号的权限
调度器	定时或事件触发	以自身身份	无用户上界；记录任务创建者
Agent	被托管的执行单元	由调用方的模式决定	能力声明、授权记录、硬性禁止项共同约束

调度器是一个容易被忽略的主体。定时任务触发时没有“当前用户”，那么权限的上界取谁？本方案的规则是：定时任务以 Agent 自身身份运行，权限上界是能力声明与授权记录的交集，同时记录任务的创建者作为责任归属；每次触发前检查创建者仍然在职、Agent 未被禁用；创建者离职后，其定时任务随 Agent 一起冻结并进入复审。这条规则防的是“无人负责的自动化”长期运行。

每个 Agent 必须有人类责任人（owner）；责任人离职触发名下 Agent 的复审。注册表里的条目就是 Agent 的身份蓝图，Agent 本体不持有任何长期凭证。

14.2 有效权限的交集规则

一次工具调用能不能执行，由下面的交集决定：

$$P_{\text{有效}} = P_{\text{能力声明}} \cap P_{\text{工具授权}} \cap \begin{cases} P_{\text{用户}}, & \text{代表用户} \\ P_{\text{服务}}, & \text{以服务身份} \\ \Omega, & \text{以自身身份（调度器，不再收窄）} \end{cases} \setminus P_{\text{硬性禁止}}$$

读法是：Agent 自己声明能用的，交上管理员授权它用的，再交上发起方本身有权做的（调度器没有“本身的权限”，这一项取全集 Ω ，即不再收窄），最后去掉硬性禁止项。三个集合取交集意味着任何一方都只能收窄权限，不能放大——一个 Agent 代表用户行动时，权限不会超过这个用户自己的权限；一个 Agent 被授予了某个工具，如果它的能力声明里没有，也用不了。

硬性禁止项是一份对 Agent 类主体永远不开放的清单（例如修改自身权限、跨租户访问），它不进策略配置，在身份层就不存在授予它的路径。

每一条事件都携带完整的身份链：租户、Agent、版本、执行环境、行动模式、代表谁、审批人。审计时沿着这条链就能回答“这次操作是谁、代表谁、经谁批准做的”。

14.3 令牌分两类，各走各的接口

平台发的令牌分两种，各自只能用于一类接口（表 23、图 16）。

表 23 两种令牌

令牌	用于哪类接口、谁持有	限制
平台令牌	统一 API；持有者是人、CI 和业务服务	不能访问模型网关、工具网关、检查点等运行侧出口
运行令牌	模型网关、工具网关、检查点、产物；持有者是运行中的容器	绑定单次执行，分钟级有效，限制来源网段，不能访问统一 API

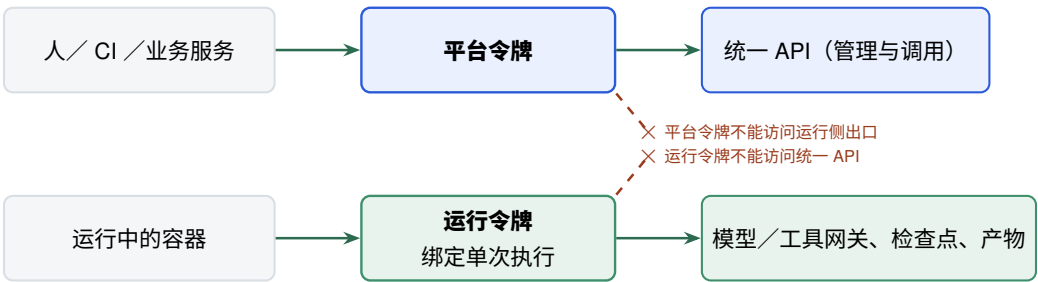


图 16 两种令牌各走各的路：交叉使用一律被拒绝

这个设计把“管理平台的权限”和“代表一次执行去行动的权限”在结构上隔开。容器里的代码即使拿到了自己的运行令牌，也不能用它去调管理接口修改授权；反过来，管理员的令牌也不能冒充某次执行去调工具。任何一类令牌泄露，都不会直接换来另一侧的能力。

14.4 “代表用户”必须可验证

当业务系统替某个用户发起调用时，它要告诉平台“我代表用户 U”。这个声明如果只是请求里的一个字符串，那么任何业务系统都可以冒充任何用户，前面的权限交集和审计归因就全部失去意义。所以本方案要求：“代表谁”必须是由企业身份提供方签发、经平台验签的令牌，或者是平台自己做令牌交换得到的凭证。请求头里的裸用户 ID 不能进入身份链。

14.5 工具的风险分级

工具网关维护一份四级风险字典：读取内部数据、写入内部数据、对外发送、高风险写操作。分级有两个直接用途：问答型 Agent 默认不能获得对外发送类工具；同时持有“读取敏感数据”和“对外发送”两类授权的 Agent，授权时需要两人签核，审计中对这种组合打标记。这是对前面提到的“困惑代理”风险的降低手段。

14.6 凭证保险库

工具需要的密钥（API 密钥、账号口令等）统一存放在凭证保险库里。保险库有三条铁律：

1. 人只能写入、轮换和吊销凭证，不能读出明文；
2. 明文读取路径只对工具网关的服务身份开放，保险库在网络上也只有工具网关可达；
3. 密钥永远不进入模型的上下文，也不出现在工具的返回值、日志和快照里。

图 17 展示一次工具调用中凭证的流转：Agent 声明要调用某个工具，工具网关校验权限后从保险库短时取出凭证，代替 Agent 去调用目标系统，再把结果返回给 Agent。Agent 从头到尾看到的只是工具的名字和结果。

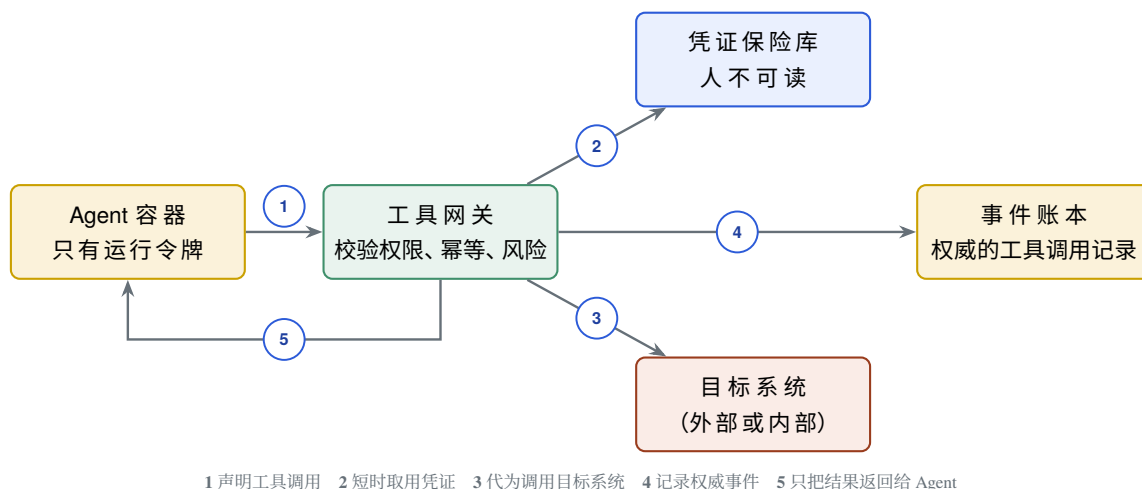


图 17 一次工具调用中凭证的流转：凭证只在工具网关短时出现，从不进入容器

15

公共服务层：副作用的统一出口

15.1 为什么需要统一出口

Agent 对外部世界的一切影响，都发生在三类动作里：调用模型、调用工具、执行代码。如果这三类动作可以从容器里直接发出，那么平台既不知道它们发生了，也无法阻止它们。公共服务层的作用，就是让这三类动作各自只有一条路可走，并且这条路上有平台的检查、记录和拒绝能力。这一层与入口层方向相反：入口层是外部调用平台，公共服务层是运行中的 Agent 反过来调用企业能力。

图 18 是全景图里的公共服务层：四个模块，两个复用企业已建的网关，两个自研。

15.2 模型网关

模型网关是全部大模型调用的唯一出口。它负责：模型白名单、用量预算（超限即预算超限终态）、逐调用计量、多模型路由、数据分级到模型白名单的映射（涉及敏感数据的租户只能使用指定的模型域）、以及 Agent 黑名单（三道停机闸门的第二道）。第一阶段至少提供一条全局的数据分级



图 18 公共服务的四个模块：Agent 只直接接触两个网关，凭证保险库和沙箱服务由工具网关代为调用（全景图中的公共服务层区域）

默认规则，更细粒度的内容防泄露在第二阶段演进。云厂商自带的模型网关不进入权威链路——原因是它握着“能不能用某个模型”的决定权，也托管着模型密钥。

15.3 工具网关

工具网关是全部工具调用的唯一出口，也是权威工具调用事件的来源。本文用“工具网关”作为统一称呼；部分接口沿用了 Tool Hub 这个名字（例如环境变量 `TOOL_HUB_URL`），只作为兼容别名保留。

工具分四类，只有最后一类需要沙箱：

1. **技能说明**。以文本形式装进 Agent 上下文的能力描述，不涉及外部调用。
2. **API 工具**。直接调用某个目标业务服务。
3. **MCP 工具**。由独立的工具服务容器提供，通过模型上下文协议接入。
4. **环境交互工具**。执行代码、运行命令、操作浏览器或桌面。这类工具需要一个隔离的临时环境。

环境交互工具的调用链路如图 19：Agent 请求工具网关，网关请求平台的沙箱服务，沙箱服务通过云适配层向供应商申请一个隔离环境。平台内部的组件之间也走平台自己的抽象层，不直接穿透到供应商。

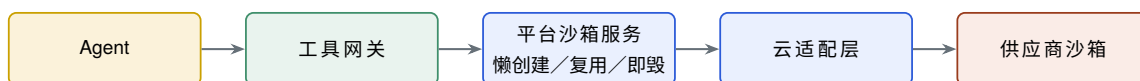


图 19 环境交互类工具的调用链路：平台内部组件之间也走平台自己的抽象，不穿透供应商

第一阶段在企业已有工具网关上做的增量包括：四类工具的分流、向容器注入网关地址、幂等台账、Agent 黑名单和最小的风险字典。

15.4 沙箱服务

沙箱服务是平台自研的轻量组件，负责隔离环境的生命周期：需要时才创建、同一会话内复用、用完即销毁、出网收敛到受控出口。它通过独立的 `/v1/sandboxes` 接口开放，工具网关只是它的消费方之一，租户也可以直接申请沙箱执行代码。凭证不进入沙箱；沙箱里的代码需要访问受保护资源时，仍然通过工具网关代为调用。

15.5 信任边界声明

容器里的 SDK、自主型的垫片、Agent 自己上报的事件，都不是信任根。安全边界由企业的网关、受控的网络出口、短时身份、沙箱隔离和企业侧的账本共同构成。

16 统一账本、计量与可观测性

16.1 账本是什么

统一账本是企业自己持有的、只追加的事件记录。Agent 的每一次执行、每一次工具调用、每一次策略判定、每一次管理操作，都以带序号和身份链的事件形式记入。它同时服务三个消费方：用户界面从中读取事件流，审计从中回放责任链，计量从中计算成本。三方消费的是同一份数据，所以它们之间不会出现不一致。

16.2 三类事实各有权威来源

同一份事件流可以被多方消费，但不同的用量数据必须尊重各自的权威来源（表 24）。这个原则的核心是：容器自己上报的数据只能作为补充，不能作为计费 and 审计的依据。

表 24 各类事实的权威来源

事实	权威来源	原因
模型用量	模型网关	掌握真实的请求与响应计量
工具调用与副作用	工具网关	处于授权、幂等和外部调用的出口
运行时忙时	供应商账单	决定企业实际支付的金额
沙箱时长	平台沙箱服务加供应商	平台掌握申请语义，供应商掌握资源账单
容器自报	仅作参考	容器不是可信的计量与审计来源

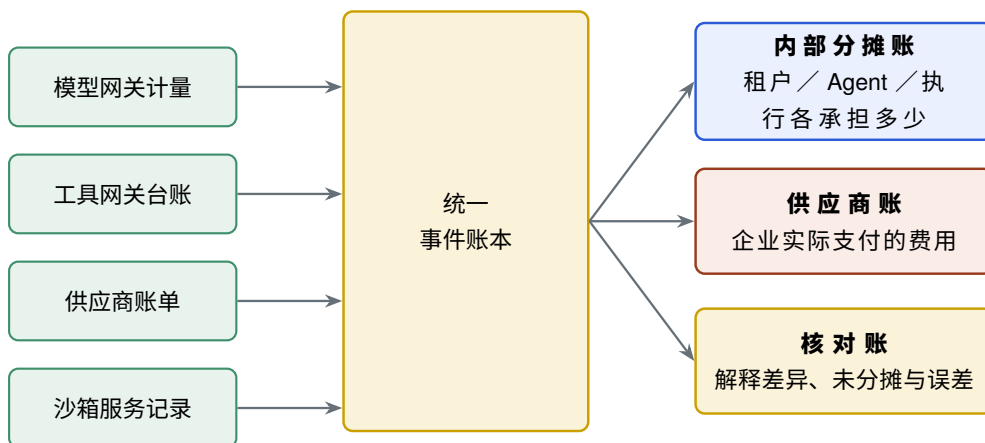
16.3 计量三本账

计量分三本账（图 20）：内部分摊账回答哪个租户、哪个 Agent、哪次执行应承担多少成本；供应商账是企业实际付给模型、云和工具供应商的钱；核对账解释两者之间的差异、未能分摊的成本和估算误差。三本账都从同一份事件流算出。

两条实现要求。第一，熔断的金额上限和计量的估算必须使用同一个计数器，否则会出现“预算系统认为没超限、任务引擎却还在跑”的分裂。第二，对账的粒度取决于供应商：如果供应商账单支持按会话或实例打标签，就能按执行对账；如果不支持，只能到租户级，文档必须如实说明，不能夸大。

问答型共享池的成本，按各租户的活跃时长占比分摊：

$$C_{\text{租户, 共享池}} = C_{\text{共享池, 忙时}} \times \frac{T_{\text{租户, 活跃}}}{\sum_i T_{i, \text{活跃}}}$$



一次执行的完整成本包含七个维度：

$$C_{\text{执行}} = C_{\text{模型}} + C_{\text{工具}} + C_{\text{运行时}} + C_{\text{沙箱}} + C_{\text{存储}} + C_{\text{网络}} + C_{\text{平台}}$$

16.4 运营指标

作为参考架构，本文不预设未经实测的成功率、延迟或可用性目标。实施之后至少应同时测量六项指标：成功完成的生产执行数、单次执行成本、策略拒绝率、审计事件完整率、人工接管率、供应商账单差异。正式的服务等级目标应在容量压测、故障演练和供应商能力验证之后，按业务等级确定。单独看“接入了多少 Agent”或“调用了多少次”容易掩盖失败、越权和浪费。

16.5 观测的三层

可观测性分三层，每层的建设方式不同：

- 1. 埋点。**平台代码统一采用 OpenTelemetry 开放标准，每个工具调用对应一个追踪片段，链路追踪标识全程传播。埋点在企业自己的代码里，一次装好。
- 2. 收集。**容器的标准输出和错误输出先进入平台的收集器，脱敏后再发送到观测后端；供应商原生的日志留存设为最短或关闭。这条规则防的是提示词和用户数据以日志形式留在供应商侧。
- 3. 面板。**第一阶段可以租用云厂商的观测后端，因为埋点走的是开放标准，换一家只改数据地址；但租户只能通过平台查看自己的追踪，不直接打开供应商面板。

默认的追踪属性使用白名单：提示词、模型输出和工具的敏感参数不进入面板。监控回答的是“机器是否健康”，账本回答的是“谁做了什么”；责任追溯永远以企业账本为准。

16.6 防篡改与合规删除

账本只追加，每条事件包含前一条事件的哈希值，形成哈希链；每日的链摘要锚定到带版本锁定或一次写入不可改的存储介质。修改任何一条历史事件都会导致链校验失败，这是审计可以信赖账本的技术基础。

合规删除（例如用户行使删除权）与只追加账本看起来矛盾，解决方式是按对话线程加密：每个线程的事件正文用独立密钥加密，需要删除时销毁密钥，正文即不可读，而事件的顺序、类型和审计结构保持完整。

17

权威存储底座

17.1 逻辑上一个库，语义先行

第一阶段用一个逻辑上的 PostgreSQL 承载事件账本、检查点、注册表和对话线程与执行环境的映射。这样做是为了先把统一的数据语义冻结下来，而不是把所有负载永久塞进一个数据库。将来把事件流拆到消息队列、把分析拆到列式存储、把检查点拆到专用存储时，上层的契约都不需要改变。

图 21 是这个逻辑单库承载的四类数据。它们的共同点是“权威”：任何模块要回答“真实发生了什么”，都以这里为准。



图 21 权威存储底座承载的四类数据（全景图中的存储底座区域）

17.2 容量与写入量治理

输出片段（message.delta）是最主要的写入量放大来源。做一个粗略估算：高峰每小时 2,000 次对话，每次约 100 个片段，就是每小时 20 万行，30 天接近 1.5 亿行；再加上长任务每一步几百 KB 的检查点，单个数据库会先被容量而不是性能拖垮。所以第一阶段就要采用：

- 输出片段进入短期保留的流表，默认保留 7 天；
- 执行完成后把片段合并成长期保留的消息事件；
- 检查点默认保留 7 天，执行完成后缩短到 24 小时；
- 大对象通过对象存储引用，不在数据库里内联；
- 数据库采用主备、静态加密、按时间点恢复和异地每日备份。

数据恢复目标必须在容量压测、备份恢复、区域故障和供应商失联演练之后再确定。本文不预填未经验证的恢复点和恢复时间目标。

18 执行平面与云供应商边界

这一节对应全景图里最下面的两层（图 22）：执行平面里的壳和适配层是平台的，两条产品线上的机器是租的；再往下的云端资源全部按量租用。这两层之间有一条分界线——供应商的名字只出现在最底层，执行平面里没有任何模块直接依赖某一家供应商。



图 22 执行平面与云端资源（全景图最下面的两层）

18.1 云适配层：五个基本操作

平台只通过五个基本操作使用云资源：部署、调用、取消、查询、销毁。平台的其他部分不直接依赖任何具体供应商。这五个操作的接口形状就是多云的预留：换供应商时，只需要实现一个新的适配器。

有三项能力虽然由供应商的运行时提供，也必须被适配层抽象掉：会话保持、版本切流、忙闲上报。它们是供应商各自的“方言”，不同供应商的语义并不相同，让它们穿透到任务引擎会在换供应商时造成隐性返工。

18.2 存活探测与忙闲的权威判断

容器的 `/ping` 只代表“还活着”。忙闲的权威判断来自任务引擎：这个执行环境上有没有未结束的执行。云适配层据此向供应商上报忙闲。这样设计有两个目的：防止容器通过虚报“忙”让供应商永不回收（空转计费），或虚报“闲”让供应商回收掉正在执行的实例；同时保证如实上报空闲，这是供应商按忙闲计费能真正省钱的前提。能力声明里的最长运行时长由平台的计时器强制：到期先发取消，宽限期后直接销毁，不依赖容器配合。

18.3 两条产品线

执行平面只有两条租用的产品线（表 25）。供应商的名字只作为租用来源标注，不作为执行平面的组件。

表 25 执行平面的两条产品线

产品线	承载内容	计费与隔离
运行时 Runtime	问答型运行壳池、代码型 SDK 壳、流 程执行器	托管持续运行，按忙闲计费
沙箱 Sandbox	工具级的代码与浏览器执行、自主型整 机环境	临时计算，按次或按时使用，用完即毁

18.4 供应商能力的三档取舍

用第 4 节的三条判据（不握决定权、不托管权威数据、有标准出口）衡量供应商的全套产品，结论分三档（表 26）。

表 26 供应商组件的三档取舍

结论	组件	理由
使用	运行时、沙箱、对象存储、只读的观测 面板	提供机械资源，不握最终决定权；观测采用开放标准，可 替换
不使用	云上的模型网关与工具网关、凭证托管、 身份服务、知识库、记忆服务、低代码 平台、托管护栏	握有决定权、托管密钥或权威数据，形成深度锁定
备选	第二家的运行时、专项沙箱产品	通过适配器和能力矩阵接入，不改变上层契约

18.5 采购的一票否决项

供应商验证至少覆盖六项：实例级或网络级的出网收敛、会话亲和、虚拟机快照、账单标签的粒度、原生日志能否关闭或缩短留存、忙闲语义能否与平台的判断对齐。任何一项影响十条不变量或退出能力时，应作为采购的否决条件，而不是上线后的工程补丁。

18.6 多云不是双倍部署

第二阶段的多云调度层负责能力矩阵、小流量试探、熔断改道与跨云重放。规则是“一次执行环境在生命周期内固定在一家供应商”，避免跨云共享内存状态；容灾依赖企业自持的检查点、产物和账本——在另一家云重新创建执行环境并从存档重放，而不是维持昂贵的热双活。

PART IV

产品化、交付与演进

把架构变成可复用的产品：企业系统怎么对接、内核与策略包怎么分、第一阶段交付什么、按什么验收、下一步是什么。

19 企业系统对接与通用化

19.1 复用企业已有系统

平台不是孤岛。企业通常已经有身份提供方、模型网关、工具网关、持续集成流水线、财务系统和事件总线，这些都按“复用、对接”的方式接入，不重建（表 27）。对于还没有模型网关或工具网关的企业，平台提供默认实现（开源的大模型网关加内置的工具网关）。图 23 是全景图里企业内部系统那一层，每个系统标出了它对接到平台的哪个模块。



图 23 企业内部系统及其对接的平台模块（全景图中的企业内部系统区域）

表 27 企业内部系统的对接方式

企业系统	对接的平台模块	方式	阶段
身份提供方	身份与访问管理、控制台、命令行	人与 CI 的登录、用户委托验签、服务身份	第一
模型网关	公共服务层	增加 Agent 黑名单和数据分级映射	第一
工具网关	公共服务层	增加四类分流、幂等台账和风险字典	第一
CI 与依赖扫描	注册与发布	代码型镜像构建时复用	第一
财务分摊系统	计量与对账	导出分摊账；自动入账后续	第一／第二
风控审批台	中断与恢复	推送单据，决议回传后恢复执行	第二
企业事件总线	事件触发	业务事件创建执行	第二
企业对象存储	文件与产物	可替代租用的对象存储	可选

19.2 内核、适配包与策略包

要让同一套架构在不同企业、不同行业复用，必须先说清哪些部分不变、哪些部分按企业替换。本方案把平台分成三层（表 28）。

表 28 内核、适配包与策略包

层次	内容	变化频率
通用内核	三层接口、五条行动通道、四维度能力声明、任务引擎与熔断、统一账本与哈希链、身份核心、五个基本操作的适配层、三本账、三道停机闸门	跨企业不变，严格版本化
适配包	身份提供方绑定、模型网关与工具网关的对接、云适配器、观测后端	随企业已有系统和供应商替换
策略包	风险字典、审批规则、数据分级、硬性禁止项、留存与删除策略	随行业、地区与企业的风险偏好变化

内核不包含任何行业业务逻辑，这是通用化的关键。一个新企业落地只需要四步：绑定身份提供方并选择云适配器；对接已有的网关，或者使用默认实现；填写形态预设和风险字典；跑一遍十条不变量的验收和符合性测试。内核代码不需要改动。

19.3 一个虚构的策略包示例

下面的例子用来说明行业约束如何进入策略包而不污染内核；它不对应任何真实组织。某医疗服务机构允许 Agent 处理预约协调、材料预审和保险信息核对。策略包规定：受保护的健康信息只能进入指定的模型域；对外发送病历摘要必须经过人工确认；修改诊疗结论、签署处方、扩大自身权限和跨租户访问属于硬性禁止项；高风险流程只允许流程型或经专项审查的代码型 Agent 承载，并要求每步检查点、冷中断和完整审计链。

示例边界：本节是虚构的策略包，不构成医疗、隐私或合规建议。正式实施必须由所在地区的法务、安全和业务责任人重新定义数据分级、审批条件和禁止项。

20

第一阶段交付、演进路线与验收

20.1 第一天必须做对的三件事

- 有三件事一旦做错，事后修改的代价是重写全部已上线的 Agent，所以必须在第一天做对：
1. 契约冻结。六种事件的格式，连同中断事件和恢复字段，一次定义清楚。
 2. 权威状态自持。事件、检查点、注册表和会话映射从第一天起就落在企业自己的数据库里。
 3. 双网关在链。模型调用和工具调用从第一天起就没有绕开网关的路径。

20.2 分阶段范围

演进顺序是“先冻结不可返工的契约，再扩大治理能力，最后开放更高的自治”（图 24、表 29）。阶段之间的边界不是功能多少，而是风险边界逐级放开的门槛。

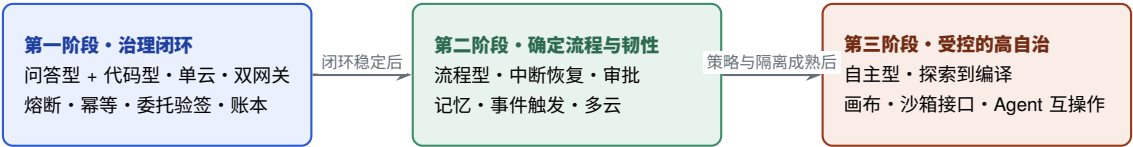


图 24 从治理闭环到受控高自治的演进路径

表 29 三个阶段的范围

阶段	核心能力	产品边界
第一阶段	问答型与代码型、单云、定时触发、简版控制台、熔断、两类幂等、两类令牌隔离、用户委托验签、三道停机闸门、哈希链、日志归口、输出片段流表	建立从开发到安全追责的完整闭环
第二阶段	流程型、中断执行机制、审批工作台、配额管理、长期记忆、事件触发、多云调度、自建观测面板、高级控制台	增强确定性流程与运行韧性
第三阶段	自主型、探索到编译的管线、流程画布、沙箱接口产品化、Agent 互操作	开放更强的自治，同时保持治理边界

20.3 六个验收闭环

第一阶段怎么算建成？不是“组件都部署了”，而是六条端到端的路径都能从入口跑到最终证据（表 30）。组件清单用于建设，闭环用于验收。

表 30 第一阶段的六个验收闭环

#	闭环	端到端验收
1	开发上线	少量改造、云端构建、发布；符合性检查通过后上线，并且可以回滚
2	对话服务	提问或定时触发，实时流式输出，断线后按序号续读
3	工具执行	网关鉴权、短时凭证注入、四类分流，环境交互类工具按需开沙箱
4	审计追责	账本完整回放、身份链归因、管理动作关联、哈希链校验通过
5	成本对账	按 Agent 和执行的分摊可以解释，与供应商的忙闲账单完成核对
6	安全闭环	失控的循环被熔断、越权的调用被拒绝、重复创建返回同一个执行、伪造的用户委托被拒绝、供应商失联时前两道停机闸门仍然生效

21

关键架构决策与取舍

表 31 汇总本文最重要的十二项决策，每项都写明得到了什么、付出了什么。评审时可以逐项质疑。

表 31 关键架构决策

编号	决策	得到	代价与约束
A1	用协议适配框架	框架可替换，契约稳定	需要维护适配器和符合性测试套件
A2	对外完整、对内精简	产品体验与运行稳定解耦	控制平面承担更多语义
A3	四个维度取代形态硬编码	新场景靠组合表达，策略正交	能力声明与准入校验更严格
A4	先记账再分发	断线续读、事件有序、可回放	账本成为关键写路径
A5	网关事件为权威事实	不依赖不可信的容器自报	网关必须高可用、低延迟
A6	熔断进入第一阶段	首次上线就有损失上界	增加计数与终止逻辑
A7	令牌分两类，各走各的接口	管理面与运行面在结构上隔离	令牌交换与接口校验更复杂
A8	三道停机闸门	停机不依赖云供应商	需要注册表与双网关联动
A9	单库起步、语义先行	快速交付且语义一致	必须提前治理输出片段带来的写入量
A10	单云落地、契约多云	控制初期的运维面	迁移能力需要持续演练来证明
A11	自主型延后到第三阶段	先解决主流、可治理的工作负载	暂不覆盖最高自治的场景
A12	策略包隔离行业逻辑	内核可以跨行业复用	每个行业要维护自己的策略资产

这些决策背后有一个统一的顺序：先建立身份和账本，再扩展记忆和智能调度；先验证最小权限和退出路径，再开放更多工具和更高的自治。

22

残余风险与后续规范

22.1 明确接受、持续降低的风险

有些风险本方案降低了但没有消除，表 32 如实列出，评审和发布时应主动说明。

22.2 配套规范

本文是架构层面的描述。字段级的定义由三份配套规范承载，并与本文分开维护、独立编号：容器契约的字段级规范（序号、事件去重标识、存活探测语义、格式版本、错误类别）发布为 OpenAPI 与 JSON Schema；工具授权模型与身份的详细设计；符合性测试套件，至少包含进程强制终止、执行环境回收后重建、出网封锁三个用例。三者之中，容器契约的规范优先级最高，因为它不可返工——它没有冻结之前，不应并行推进多个 Agent 框架的接入。

表 32 残余风险

风险	降低手段	为什么仍然残余
跨工具的困惑代理	风险分级、敏感读取与对外发送的组合双签、参数策略	合法权限的组合仍可能被注入指令利用
文件内容里的隐式指令	内容隔离、扫描、工具最小权限	非结构化文件可能携带恶意指令
经外部模型的数据泄露	数据分级到模型白名单、日志脱敏	第一阶段缺少细粒度的内容防泄露
供应商对账粒度	标签验证、内部三本账	粒度最终受供应商账单能力限制
供应商可见运行明文	出网收敛、最小数据、合同与区域约束	托管计算天然需要处理明文

23

结论

企业级 Agent 平台的工程范围远大于“把 Agent 放进容器”。模型决定 Agent 怎么推理，运行时决定它在哪里运行，控制平面决定它以谁的身份、在什么边界内、为谁承担成本和责任。

本文提出的架构，用协议代替框架绑定，用四个维度的能力声明代替形态硬编码，用执行环境、对话线程、执行三个概念分开资源、对话和执行，用五条行动通道封闭副作用，用两类令牌隔离、用户委托验签、凭证保险库和三道停机闸门构成身份安全，用统一账本连接事件、审计、计量和恢复，再用五个基本操作的云适配层保留供应商选择权。

这套设计的关键不在组件的数量，而在边界是否可以证明：长期密钥是否从未进入容器，副作用是否没有旁路，事件是否先记账，权限是否只能收窄，成本是否有硬上限，供应商失联时 Agent 是否仍然能被切断。这十条不变量成立，平台才真正拥有控制权。

对企业而言，这意味着三件具体的事：业务团队可以更快地把 Agent 用起来，因为接入、算力和权限不需要每个团队自己解决；安全和审计部门可以放心地批准它进入生产，因为每一次行动都有出口、有记录、有上限；管理层不必为了速度把控制权交给某一家供应商，因为契约之下的一切都可以替换。

企业可以租用智能体运行所需的算力和环境；身份、钥匙、决定权、事实和退出权，应该留在自己手里。

附录 A

术语表

按首次出现的顺序排列，第一次接触的读者可以先通读一遍。

术语	含义
Agent	一个可以版本化、可以赋予身份、能在一次执行中根据输入选择模型或工具并产生结果的软件单元。不是一个模型，也不是一段提示词。
可控／可计量／可观察／可审计	平台的四类目标能力：能限制 Agent 做什么；能算清每次执行花了多少；能看到系统健不健康；能回放谁在什么时候做了什么。
控制平面／执行平面	控制平面是企业自己持有的决策与记录部分（身份、权限、任务、账本）；执行平面是 Agent 实际运行的可替换的算力部分。
运行壳（harness）	平台自己实现的通用执行循环，负责调用模型和编排工具。问答型 Agent 都跑在它上面。
T0 ／ T1 ／ T2 ／ T3	四种 Agent 形态，按“下一步由谁决定”划分：问答型“照本回答”（平台固定循环）、流程型“按单办事”（事先画好的流程图）、代码型“自带程序入驻”（租户自己的代码）、自主型“请专家自己动手”（现成的强自主 Agent）。
能力声明清单（manifest）	每个 Agent 注册时提交的声明，包含四个维度的取值、是否流式、最长运行时长和运行壳版本。
四个维度	能力声明的四个相互独立的维度：循环归属、隔离等级、副作用范围、恢复能力。
符合性认证	能力声明通过破坏性测试（进程强制终止后恢复、执行环境回收后重建、出网封锁）后才生效的机制。
L1 ／ L2 ／ L3	三层接口：对外的统一 API；平台与容器之间的最小契约；容器调用企业服务出口。
执行（Run）	一次有开始、有终态、有预算、可以取消的执行，是计量和审计的基本单位。
对话线程（Thread）	一段长期的对话历史，可以跨多个执行环境和多次执行。
执行环境（Session）	一块隔离执行环境的短期租约，活跃时占用机器，空闲时可以释放。
检查点	执行过程中保存的安全点，失败后可以从这里继续而不是从头重做。
中断与恢复	Agent 执行中途需要等待（等人审批、等外部回调）时，存好状态、释放机器，结果回来后从存档继续。
行动通道	运行中的 Agent 对外部世界产生影响的五条路径：被调用、调用模型、调用工具、请求人工输入、调用其他 Agent。
幂等	同一个操作重复执行不会产生额外效果。平台有创建幂等（重复请求不会创建多个执行）和副作用幂等（重试不会把真实动作做两遍）两类。
熔断	每次执行在步数、调用次数、模型用量、时长、金额上的硬上限，任一超限即终止。
等效恰好一次	通过“副作用前保存检查点、工具网关幂等台账、结果存档”三件事组合出的语义：真实世界的动作不会因为重试而重复。
三道停机闸门	停止签发令牌、网关拒绝该 Agent、销毁实例；前两道不依赖云供应商。
用户委托（OBO，on-behalf-of）	业务系统替某个终端用户发起调用；“代表谁”必须是身份提供方签发的令牌。
身份提供方（IdP）	企业统一的登录与身份签发系统，例如单点登录服务。

术语	含义
平台令牌／运行令牌	两类令牌：前者给人、CI 和业务服务用于统一 API；后者给运行中的容器用于模型、工具等出口，绑定单次执行、分钟级有效。
权限交集规则	有效权限等于能力声明、工具授权、发起方权限三者的交集，再去掉硬性禁止项；任何一方只能收窄，不能放大。
硬性禁止项	对 Agent 类主体永远不开放的操作清单，在身份层就不存在授予它的路径。
凭证保险库	集中保管工具密钥的服务：人只能写入、轮换、吊销，不能读出；只有工具网关能短时取用。
模型网关／工具网关	全部模型调用和全部工具调用各自的唯一出口，也是平台检查、记录、拒绝的位置。
沙箱	用完即毁的隔离执行环境，用于跑代码、操作浏览器等环境交互类工具。
统一账本	企业自持的、只追加的事件记录，同时服务界面、审计和计量。
哈希链	每条事件包含前一条的哈希值，修改任何历史事件都会导致校验失败。
三本账	内部分摊账、供应商账、核对账。
云适配层	把平台的五个基本操作（部署、调用、取消、查询、销毁）翻译成供应商接口的防腐层。
内核／适配包／策略包	跨企业不变的部分；随企业已有系统和供应商替换的部分；随行业 and 风险偏好变化的部分。
OpenTelemetry	可观测性数据的开放标准，规定了埋点方式、数据格式和传输协议；采用它之后更换监控后端只需改数据地址。
MCP	模型上下文协议，一种把工具接入大模型应用的开放标准。

附录 B

L2 容器契约速查

类别	项目	语义
端点	POST/invoke	启动或恢复一次执行，返回服务器推送的事件流
端点	GET/ping	仅作存活探测；忙闲以任务引擎的判断为准
端点	POST/cancel	请求在安全点退出；宽限期后由云适配层销毁
事件	message.delta	流式输出片段，短期保留后合并为消息
事件	tool.call	容器声明的工具调用；以工具网关记录为权威
事件	checkpoint	执行恢复点，带能力等级
事件	interrupt	中断请求单据；格式第一阶段冻结，机制第二阶段实现
事件	artifact	大对象的引用，不内联内容
事件	run.status	状态、错误类别、运行壳版本
环境变量	MODEL_GATEWAY_URL	模型的唯一出口地址
环境变量	TOOL_HUB_URL	工具的唯一出口地址
环境变量	CHECKPOINT_STORE	检查点服务地址

类别	项目	语义
环境变量	ARTIFACT_STORE	文件与产物服务地址
能力声明	四个维度、流式、最长时长、壳版本	声明能力、限制和运行壳版本；经符合性认证后生效
错误类别	retryable / terminal / policy_denied / budget_exceeded	可重试、不可重试、策略拒绝、预算超限

附录 C

十条不变量验收清单

#	验收项	状态
1	容器、镜像、日志和快照中不存在长期密钥	☐
2	从容器内直连模型、工具或外网均失败，副作用无法绕过统一出口	☐
3	工具调用的审计以工具网关事件为准，与容器自报可比对	☐
4	企业侧账本只追加，哈希链可验证，篡改历史事件会被发现	☐
5	每次执行的步数、调用次数、模型用量、时长和金额均有硬上限并可触发	☐
6	平台令牌与运行令牌交叉使用均被拒绝	☐
7	伪造的用户委托无法通过身份提供方验签	☐
8	模拟供应商接口不可用时，前两道停机闸门仍然生效	☐
9	事件序号由平台分配，客户端断线后能凭序号续读	☐
10	管理动作与 Agent 动作能在同一条身份链中回放	☐

附录 D

修订记录

日期	版本	说明
2026-09-17	设计基线 v3.0	内部最终设计文档与总体架构图定稿
2026-09-18	技术白皮书 v1.0	面向公开发布的参考架构定稿：增加相关工作、一次调用全过程、威胁模型、总体架构全景与各章局部图、术语表；统一术语；去除未经实测的数字

参考文献

- [1] Amazon Web Services. *Amazon Bedrock AgentCore FAQs*.
<https://aws.amazon.com/bedrock/agentcore/faqs/> (访问日期: 2026 年 9 月)
- [2] Anthropic. *Claude Platform Documentation: Managed Agents*.
<https://platform.claude.com/docs/en/home> (访问日期: 2026 年 9 月)
- [3] OpenAI. *Agents SDK and Agents Platform Documentation*.
<https://developers.openai.com/api/docs/guides/agents-sdk> (访问日期: 2026 年 9 月)
- [4] Google Cloud. *Gemini Enterprise Agent Platform: Agent Runtime*.
<https://docs.cloud.google.com/gemini-enterprise-agent-platform/build/runtime>
(访问日期: 2026 年 9 月)
- [5] Microsoft. *Configure agent identity security with the Zero Trust Assessment*.
<https://learn.microsoft.com/en-us/security/zero-trust/assessment/ai-pillar> (访问日期: 2026 年 9 月)
- [6] LangChain. *LangSmith Deployment*.<https://docs.langchain.com/langsmith/deployment> (访问日期: 2026 年 9 月)
- [7] Temporal. *Durable AI*.<https://docs.temporal.io/ai> (访问日期: 2026 年 9 月)
- [8] Tencent Cloud. *CubeSandbox: Secure MicroVM Sandbox for AI Agents*.
<https://github.com/TencentCloud/CubeSandbox> (访问日期: 2026 年 9 月)